VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belagavi-590018, Karnataka, India

A PROJECT REPORT ON

"CliniScan+ : An Integrated AI – Based Lung Disease Detection Diagnosis and Smart Healthcare Support System"

Submitted in partial fulfilment of the requirement for the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE ENGINEERING

Submitted By

Rishika Sharma (1RG23CS053)

Shiva Kumar (1RG23CS064)

Vedashree R (1RG23CS079)

Yudhil Krishna (1RG22CS092)

Under the guidance of

Dr. Latha P H

Professor

Dept. of CSE, RGIT

Department of Computer Science & Engineering

RAJIV GANDHI INSTITUTE OF TECHNOLOGY

Cholanagar, R.T. Nagar Post, Bengaluru-560032

2026-2027

RAJIV GANDHI INSTITUTE OF TECHNOLOGY

(Affiliated To Visvesvaraya Technological University)

Cholanagar, R.T. Nagar Post, Bangalore-560032

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

CERTIFICATE

This is to certify that the Project Report entitled **"CliniScan+ : An Integrated AI – Based Lung Disease Detection Diagnosis and Smart Healthcare Support System"** is a Bonafide work carried out by **Rishika Sharma (1RG23CS053), Shiva Kumar (1RG23CS064), Vedashree R (1RG23CS079) and Yudhil Krishna (1RG22CS092)** in partial fulfillment for the award of **Bachelor of Engineering in Computer Science Engineering** under **Visvesvaraya Technological University, Belagavi,** during the year **2026-2027.** It is certified that all corrections/suggestions given for Internal Assessment have been incorporated in the report. This project report has been approved as it satisfies the academic requirements in respect of project work prescribed for the said degree.

Signature of Guide	Signature of HOD	Signature of Principal
Dr. Latha P H	**Dr. Arudra A**	**Dr. D G Anand**
Professor	Head of Department, CSE	Principal, RGIT
Dept. of CSE,	Dept. of CSE,	RGIT, Bengaluru - 32
RGIT, Bengaluru - 32	RGIT, Bengaluru - 32	

Name of the Examiners	**Signature with date**
1.	
2.	

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belagavi-590018

RAJIV GANDHI INSTITUTE OF TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

DECLARATION

We hereby declare that the Project Report entitled **"CliniScan+ : An Integrated AI – Based Lung Disease Detection Diagnosis and Smart Healthcare Support System"** submitted to the **Visvesvaraya Technological University, Belagavi** during the academic year **2026-2027**, is record of an original project done by us under the guidance of **Dr. Latha P H, Professor, Department of Computer Science and Engineering, Rajiv Gandhi Institute of Technology, Bengaluru** and this project is submitted in the partial fulfillment of requirements for the award of the degree of **Bachelor of Engineering** in **Computer Science & Engineering.** The results embodied in this Project report have not been submitted to any other University or Institute for award of any degree.

Rishika Sharma (1RG23CS053)

Shiva Kumar (1RG23CS064)

Vedashree R (1RG23CS079)

Yudhil Krishna (1RG22CS092)

ACKNOWLEDGEMENT

We take this opportunity to express our sincere gratitude and respect to the Rajiv Gandhi Institute of Technology, Bengaluru, for providing us with an opportunity to carry out this project report work.

We express our gratitude to **Dr. D G Anand**, Principal, RGIT, Bengaluru, for providing the resources and support without which the completion of this project would have been a difficult task.

We extend our sincere thanks to **Dr. Arudra A**, Associate Professor and Head, Department of Computer Science and Engineering, RGIT, Bengaluru, for being a pillar of support and encouraging us in the face of all adversities.

We would also like to express our heartfelt thanks to **Prof. Bhagyashree Wakde** and **Prof. Soniya Komal V**, Project Coordinators, for their constant guidance, encouragement, and support throughout the course of this project.

We would like to acknowledge the thorough guidance and support extended to us by **Dr. Latha P H**, Professor, Department of CSE, RGIT, Bengaluru. Her incessant encouragement and valuable technical support have been of immense help. Her guidance provided us with the environment to enhance our knowledge and skills and to reach greater heights with determination, dedication, and hard work.

We also extend our thanks to the entire faculty and support staff of the Department of Computer Science and Engineering, RGIT, Bengaluru, who have encouraged us throughout the course of our Bachelor's degree.

We would like to thank our families for always being there with full support and for providing us with a safe and encouraging environment to complete our project. We are ever grateful to them for helping us during challenging times.

Lastly, we would like to acknowledge all the valuable insights and support given to us by our friends during the course of this project report.

Rishika Sharma (1RG23CS053)

Shiva Kumar (1RG23CS064)

Vedashree R (1RG23CS079)

Yudhil Krishna (1RG22CS092)

ABSTRACT

CliniScan+ is an AI-powered clinical support system designed to detect lung diseases from chest X-ray images while providing comprehensive healthcare assistance. Unlike traditional computer-aided diagnosis systems that stop at prediction, CliniScan+ goes further by helping patients understand their results and locate appropriate doctors. The core AI engine uses an Ultralytics YOLO11 Nano object detection network trained on the Vin Bigdata Chest X-ray dataset to detect and localize ten target thoracic abnormalities: Atelectasis, ILD, Pulmonary Fibrosis, Pneumothorax, Pleural Effusion, Pleural Thickening, Consolidation, Lung Opacity, Nodule/Mass and Infiltration. Each finding is shown with a bounding box, confidence score, anatomical location and severity grading so that clinicians can verify the output. A hybrid recommendation engine combines rule-based clinical triage with Multi-Criteria Decision Analysis (MCDA) to rank specialists from a network of 2,000 doctors according to the detected pathologies, ratings, experience, verification status and availability. In addition, a 24/7 interactive Clinical Assistant powered by Google Gemini explains findings in clear, empathetic language and guides patients through specialist consultations, while a clinical tracking module maintains patient records, consultation history and appointments. Overall, CliniScan+ functions as a complete digital healthcare ecosystem that enhances early diagnosis, supports medical decision-making, bridges the gap between diagnosis and treatment, and simplifies the respiratory care journey.

CONTENTS

ACKNOWLEDGEMENT

ABSTRACT

LIST OF FIGURES

LIST OF TABLES

CHAPTER NO	TITLE	PAGE NO
1	**INTRODUCTION**	**1**
	1.1 Overview	1
	1.2 Motivation	1
	1.3 Problem Identification	2
	1.4 Scope	2
	1.5 Objectives	3
	1.6 Existing System	4
	1.7 Proposed System	5
	1.8 Background: Thoracic Abnormalities Targeted	5
	1.9 Methodology Followed	7
	1.10 Applications and Benefits	7
	1.10.1 Applications	7
	1.10.2 Benefits	7
	1.11 Report Organization	8
	1.12 Introduction Summary	8
2	**LITERATURE SURVEY**	**9**
	2.1 Enhancing Pneumonia Diagnosis Through AI Interpretability	9
	2.2 AI-Driven Deep Learning Framework for Pneumonia Detection	10
	2.3 Pneumonia Detection Using Deep Learning and Grad-CAM++	11
	2.4 CheXNet: Radiologist-Level Pneumonia Detection	11
	2.5 YOLOv11: Real-Time Object Detection	12
	2.6 VinDr-CXR: A Chest X-ray Dataset with Radiologist Annotations	13
	2.7 Literature Survey Summary	13
3	**SYSTEM ANALYSIS**	**15**
	3.1 Introduction to System Analysis	15
	3.2 Feasibility Study	15
	3.2.1 Technical Feasibility	15
	3.2.2 Economic Feasibility	15
	3.2.3 Operational Feasibility	15
	3.2.4 Ethical and Legal Considerations	15

3.2.5 Schedule Feasibility 16
3.3 Functional Requirements 16
3.4 Non-Functional Requirements 16
3.5 Software Requirement Specification 17
3.5.1 Purpose and Product Perspective 17
3.5.2 Assumptions and Dependencies 18
3.5.3 Constraints 18
3.5.4 External Interface Requirements 18
3.6 User Classes and Access Rights 19
3.7 Use Case Specifications 19
3.8 Requirements Traceability Matrix 21
3.9 Risk Analysis 22
3.10 System Analysis Summary 23

4 REQUIREMENT ANALYSIS 24
4.1 Hardware Requirements 24
4.2 Software Requirements 24
4.3 Description of Technologies 25
4.3.1 Python 25
4.3.2 PyTorch and Ultralytics YOLO11 25
4.3.3 OpenCV, NumPy and Pandas 25
4.3.4 FastAPI and Uvicorn 25
4.3.5 MongoDB 26
4.3.6 VS Code and Jupyter 26
4.3.7 Next.js, React and TypeScript 26
4.3.8 Authentication and Password Security 26
4.3.9 Google Gemini API 26
4.3.10 Git and Version Control 27
4.4 Dataset Description 27
4.5 Theoretical Background 28
4.5.1 Convolutional Neural Networks 28
4.5.2 Object Detection 28
4.5.3 YOLO11 Architecture 28
4.5.4 Intersection over Union and Non-Maximum Suppression 28
4.5.5 Confidence Threshold 29
4.5.6 Evaluation Metrics 29
4.5.7 Weighted MCDA Ranking 29
4.6 Development and Runtime Environment 29
4.7 Requirement Analysis Summary 30

5 SYSTEM DESIGN 31
5.1 Logical Design 31
5.2 Design Goals 31
5.3 System Architecture 32
5.4 Data Flow Diagrams 34

5.4.1 Level 0 DFD (Context Diagram) 34
5.4.2 Level 1 DFD 34
5.5 Use Case Diagram 35
5.6 Workflow (Activity) Diagram 35
5.7 Database Design 36
5.8 Class Diagram 36
5.9 Sequence Diagrams 37
5.9.1 AI Triage and Recommendation 37
5.9.2 Appointment Booking and Confirmation 38
5.10 State Diagram of an Appointment 38
5.11 Component and Deployment View 39
5.12 Level 2 Data Flow Diagram: AI Detection 40
5.13 User Interface Design 40
5.14 Security Design 41
5.15 Project Schedule (Gantt Chart) 42
5.16 Module Interface Specification 43
5.17 Recommendation Algorithm Flow 44
5.18 Input and Output Design 45
5.19 System Design Summary 46

6 SYSTEM IMPLEMENTATION 47
6.1 Module Description 47
6.1.1 Authentication and Role Management 47
6.1.2 Data Acquisition and Pre-processing 47
6.1.3 Model Training 47
6.1.4 Inference and Explainability 48
6.1.5 Severity Grading and Clinical Protocol 48
6.1.6 Doctor Recommendation 49
6.1.7 Gemini AI Clinical Assistant 49
6.1.8 Consultation Tracking 50
6.1.9 Report Generation 50
6.1.10 Color Coding and Presentation of Findings 50
6.1.11 Session and Role Handling 50
6.1.12 Error Handling 50
6.2 Program Code 51
6.2.1 FastAPI Inference Endpoint (src/api_server.py) 51
6.2.2 Doctor Recommendation and MCDA Scoring (src/recommendation.py) 52
6.2.3 Gemini AI Clinical Assistant Integration (src/gemini_chatbot.py) 52
6.2.4 Frontend Component: XrayScanner.tsx 53
6.2.5 Frontend Component: ClinicalChatbot.tsx 53
6.2.6 Official Diagnostics and Referral Report Generator 54
6.2.7 FastAPI X-ray Detection Endpoint (src/api_server.py) 55
6.2.8 YOLO11 Inference Service (src/inference.py) 56

6.2.9 Doctor Recommendation and Ranking (src/recommendation.py) 57
6.2.9 Gemini Clinical Assistant (src/gemini_chatbot.py) 58
6.2.10 Frontend X-ray Scanner (frontend/src/components/XrayScanner.tsx) 59
6.2.11 Frontend Clinical Chatbot (frontend/src/components/ClinicalChatbot.tsx) 60
6.2.12 Printable Referral Report Generator (app/components.py) 60
6.3 Data Processing Flow in Detail 61
6.4 Running the System 62
6.5 System Implementation Summary 62

7 TESTING 63
7.1 Testing Methods 63
7.1.1 Unit Testing 63
7.1.2 Integration Testing 63
7.1.3 System Testing 63
7.1.4 Model Evaluation 63
7.1.5 Usability and Acceptance Testing 63
7.2 Test Cases 64
7.3 Testing Summary 65

8 RESULTS AND SAMPLE OUTPUT 66
8.1 Landing Page and Login 66
8.2 Patient Portal: X-ray Upload 66
8.3 AI Detection and Localized Findings 67
8.4 Specialist Recommendation 67
8.5 Doctor Portal 68
8.6 Laboratory Portal 69
8.7 End-to-End Usage Scenario 69
8.8 Analysis of the Sample Detection 70
8.9 Summary 71

9 CONCLUSION AND FUTURE SCOPE 72
9.1 Conclusion 72
9.2 Limitations 72
9.3 Future Scope 72
9.4 Lessons Learnt 73
9.5 Social and Ethical Impact 73
9.6 Detailed Future Enhancements 73
9.6.1 Geographical Risk Analysis 73
9.6.2 Hybrid Explainability 73
9.6.3 Clinical Validation and Larger Models 74
9.6.4 Integration with Hospital Systems 74
9.6.5 Mobile and Multilingual Access 74
9.6.6 Privacy-Preserving Learning 74

REFERENCES 75

APPENDIX A: STATUS OF PAPER PUBLICATION 76

APPENDIX B: USER MANUAL 77

B.1 Patient 77

B.2 Doctor 77

B.3 Diagnostic Laboratory 77

LIST OF FIGURES / ILLUSTRATIONS

FIGURE NO	FIGURE NAME	PAGE NO
4.1	Block diagram of the YOLO11 detector	28
5.1	System architecture of CliniScan+	32
5.2	Level 0 DFD	34
5.3	Level 1 DFD	34
5.4	Use case diagram	35
5.5	Workflow of CliniScan+	35
5.6	Entity relationship diagram	36
5.7	Class diagram of CliniScan+	37
5.8	Sequence diagram of the AI triage flow	38
5.9	Sequence diagram of appointment booking and confirmation	38
5.10	State diagram of an appointment	39
5.11	Deployment view of CliniScan+	39
5.12	Level 2 DFD of the AI detection process	40
5.13	Implementation Gantt chart	42
5.14	Flow of the hybrid triage and MCDA recommendation	45
6.1	Model training pipeline	47
8.1	Landing page and role-based hospital clinical gateway	66
8.2	New account registration (prototype interface)	60
8.3	Patient portal — radiograph source and confidence threshold	67
8.4	Bounding-box detection and localized AI pathological findings	67
8.6	Prescribed specialist referral match and top recommended specialists	68
8.7	Doctor portal — patient consultation review queue	68
8.8	Confirmed patient consultation roster	69
8.9	Laboratory intake and automated AI triage	69

LIST OF TABLES

TABLE NO	TABLE NAME	PAGE NO
1.1	Existing workflow versus proposed system	4
1.2	Thoracic abnormalities and their significance	5 - 7
1.3	Benefits to each stakeholder	7
3.1	Functional requirements	16
3.1A	Measurable non-functional requirements	17
3.2	External interfaces	18
3.3	Role-permission matrix	19
3.4	UC-1 Register and sign in	19
3.5	UC-2 Run AI triage on a chest X-ray	20
3.6	UC-3 Obtain specialist recommendation	20
3.7	UC-4 Book and confirm an appointment	21
3.8	UC-5 Laboratory intake and certificate	21
3.9	Traceability of requirements	21
3.10	Project risks and mitigation	22
4.1	Hardware requirements	24
4.2	Software requirements	24
4.3	Dataset annotation format	27
4.4	Data split	27
4.5	Development environment	29
5.1	Database collections	36
5.2	Screens of the application	41
5.3	Task schedule	42
5.4	Module interface specification	43
5.6	Main input forms	45
5.7	Main outputs	46
6.1	Training configuration	47
6.2	Examples of severity and protocol mapping	49
7.1	Functional test cases	64

LIST OF TABLES (Continued)

TABLE NO	TABLE NAME	PAGE NO
8.1	End-to-end scenario	70
8.2	Findings in the sample run	71

CHAPTER 1
INTRODUCTION

1.1 Overview

Chest radiography is the most widely used first-line imaging modality for diagnosing respiratory disease. It is inexpensive, fast and available even in primary health centers, and it is used to screen for conditions such as pneumonia, tuberculosis, pleural effusion, pleural thickening and interstitial lung disease (ILD).

Deep learning has significantly improved automated detection of abnormalities on chest X-rays. Yet most published systems stop at producing a class label or a probability score.

CliniScan+ is an integrated, AI-based lung disease diagnosis and smart healthcare support system that closes this gap. A patient, an attending doctor or a diagnostic laboratory signs in to a role-based web portal and uploads a chest radiograph. A YOLO11 (Nano) object-detection model, trained on the Vin Bigdata chest X-ray dataset, localizes abnormalities across ten target pathologies, draws bounding boxes on the image, reports a confidence score and anatomical zone for every finding, and assigns a severity level with a recommended clinical protocol.

1.2 Motivation

India has a very large burden of respiratory disease, including tuberculosis and pneumonia, while specialist doctors are concentrated in a few metropolitan cities. Patients in smaller towns often have a chest X-ray taken but must wait for a report and then search on their own for a suitable pulmonologist.

Existing research on AI for chest X-rays concentrates mainly on classification accuracy. Studies of pneumonia detection with convolutional networks report very high accuracy, but often on a single binary task and without any link to the clinical workflow that follows.

Finally, the project is motivated by the need for explainable AI (XAI) in medicine. A clinician cannot safely act on an unexplained prediction. Presenting bounding boxes, confidence values and anatomical zones lets the doctor verify the AI output against the image and keeps the human in charge of the final decision.

1.3 Problem Identification

Despite the widespread use of chest X-ray imaging for diagnosing lung diseases such as pneumonia, tuberculosis and other respiratory conditions, several critical challenges persist in current healthcare systems:

- **Limited diagnostic accuracy and variability:** manual reading of chest X-rays varies between readers and is affected by fatigue, workload and image quality. Subtle findings such as early pleural thickening or interstitial change are easily missed.
- **Lack of integrated decision support:** AI tools that exist produce a label but do not tell the patient which specialist to consult, or connect the result with an appointment and a medical record.
- **Insufficient explainability in AI models:** black-box predictions without any visual evidence are difficult for doctors to trust and are unsuitable for clinical use.
- **Inadequate access to specialized healthcare:** patients, especially outside major cities, find it hard to identify and reach an appropriate pulmonologist quickly, and there is no transparent way to compare doctors by experience, rating, location and fee.
- **Fragmented records:** scans, reports, prescriptions and follow-ups are kept in separate places, so a patient's history is not available when the doctor needs it.

There is therefore a need for an intelligent, explainable and integrated healthcare system that not only detects lung diseases accurately but also provides actionable insights and doctor recommendations.

1.4 Scope

The scope of CliniScan+ covers the design, implementation and evaluation of a web-based clinical decision-support platform for chest radiographs. It includes the following aspects:

- **AI diagnosis:** detection and localization of ten target thoracic pathologies from an uploaded chest X-ray (JPEG/PNG or DICOM-derived images) using a YOLO11 Nano model trained on Vin Bigdata with a 70/15/15 train/validation/test split.
- **Explainability:** bounding boxes with disease name, confidence score and anatomical zone (for example, right hemithorax, mid zone), together with an adjustable detection-confidence sensitivity threshold.

- **Severity and protocol mapping:** conversion of each finding into a severity level (Low-Moderate, Moderate, High) and a recommended clinical protocol such as HRCT consultation or pleural assessment.
- **Doctor recommendation:** hybrid rule-based clinical triage to select the primary specialty, followed by a weighted MCDA ranking of doctors based on specialization match, rating, experience, availability, location and consultation fee.
- **Consultation tracking:** appointment booking, doctor confirmation with prescription and advice, appointment status tracking and consultation history.
- **Multi-role access:** separate dashboards for patients, attending doctors and diagnostic laboratories with authentication and role management.
- **Reporting:** an official, printable diagnostic certificate and clinical report containing patient details, findings table and annotated radiograph.
- **Out of scope:** the system is a decision-support aid, not a certified medical device; it does not replace a radiologist's report and has not been clinically validated on live hospital data. Geographical disease-risk prediction, proposed in the early planning phase, is left as future work.

1.5 Objectives

The main objective of CliniScan+ is to build an intelligent, explainable and integrated digital healthcare platform for lung disease diagnosis. The specific objectives are:

1. To develop an AI-based system for lung disease detection from chest X-ray images.
2. To improve diagnostic accuracy and speed by using a lightweight, real-time YOLO11 detector.
3. To incorporate Explainable AI (XAI) through bounding-box localization, confidence scores and anatomical zone mapping.
4. To provide actionable healthcare recommendations through severity grading, clinical protocols and specialist matching.
5. To enable a consultation tracking system covering appointment booking, doctor confirmation, prescriptions and history.
6. To build a user-friendly digital healthcare platform with role-based access for patients, doctors and diagnostic laboratories.

1.6 Existing System

In the present workflow a patient has a chest X-ray taken at a clinic or diagnostic centre. The film or image is read by a radiologist or physician, who prepares a written report. The patient then has to decide, often without guidance, which specialist to consult, find that specialist, telephone for an appointment and carry paper reports to the consultation. The main limitations of this arrangement are:

- Reporting is slow and depends on the availability of a radiologist; subtle abnormalities can be missed.
- Existing AI research prototypes usually output only a class label or a heat-map, without a precise location, severity grade or next step.
- There is no link between the diagnosis and specialist selection; patients cannot compare doctors by specialization, rating, experience, location or fee.
- Appointments, prescriptions and previous scans are kept in separate paper or digital records, so follow-up is difficult.
- Most published models are evaluated offline and are not deployed in a usable application with real users and roles.

Table 1.1: Existing workflow versus proposed system

Feature	Existing system	Proposed CliniScan+
Reading of X-ray	Manual, depends on availability of radiologist	Automatic detection within a second, reviewed by doctor
Output	Text report or AI label / heat-map	Boxes, confidence, zone, severity, protocol
Specialist choice	Patient decides unaided	Automatic specialty and ranked doctors
Appointment	Phone or walk-in	Online booking with confirmation
Records	Paper or scattered files	Central database, scan vault and certificates
Transparency	Black box (AI tools)	Every finding visible and adjustable by threshold

1.7 Proposed System

CliniScan+ is an integrated multi-module system. Its five modules work in sequence, as shown in the architecture in Chapter 5:

- **AI Diagnosis:** a CNN-based YOLO11 Nano object detector performs multi-disease detection on the uploaded chest X-ray and returns each finding with class, confidence and location.
- **Explainable AI:** bounding boxes drawn on the radiograph, with disease name and confidence, let the user and the doctor see exactly what the model detected and where.
- **Results aggregation:** individual detections are collected into a disease list with confidence scores, severity levels and a clinical protocol for each finding.
- **Doctor recommendation:** a hybrid rule-based clinical triage and weighted MCDA ranking algorithm selects the primary specialty and ranks matching doctors.
- **Consultation tracking:** appointment booking and confirmation, medical history monitoring and follow-up, stored in MongoDB.

The advantages of the proposed system over the existing one are speed (a result is available within a second of upload), transparency (every finding is visible on the image), continuity (diagnosis, referral, appointment and history are connected) and accessibility (a single web portal serves patients, doctors and laboratories).

1.8 Background: Thoracic Abnormalities Targeted

To understand the output of CliniScan+ it is helpful to recall the clinical meaning of the abnormalities that the detector is trained to localize. The following descriptions are general and are given only as background for the reader.

Table 1.2: Thoracic abnormalities and their significance

Abnormality	Description	Typical radiographic appearance	Clinical significance
Pleural effusion	Abnormal collection of fluid in the pleural space between the lung and the chest	Blunting of the costophrenic angle, a homogeneous opacity in the lower zone	May result from infection, heart failure, malignancy or tuberculosis; large

Table 1.2: Thoracic abnormalities and their significance (Continued)

Abnormality	Description	Typical radiographic appearance	Clinical significance
	wall.	with a curved upper border.	effusions cause breathlessness and need prompt clinical correlation.
Pleural thickening	Scarring or thickening of the pleural lining, often after inflammation, infection or asbestos exposure.	Linear or band-like opacity along the chest wall, often in the upper or mid zone.	Usually chronic; assessed for restrictive lung function and to exclude underlying disease.
Interstitial lung disease (ILD)	A group of disorders that cause inflammation and scarring of the lung interstitium.	Reticular or reticulonodular pattern, reduced lung volume, often bilateral.	Needs high-resolution CT (HRCT) and a pulmonologist for classification and management.
Pulmonary fibrosis	Progressive scarring of lung tissue leading to stiffening of the lungs.	Coarse reticular opacities, honeycombing in advanced cases, volume loss.	Chronic, progressive condition needing specialist follow-up.
Consolidation / Pneumonia	Air spaces filled with fluid, pus or cells, usually due to infection.	Patchy or lobar airspace opacity with air bronchograms.	Common cause of hospital admission; treated with antibiotics and monitored for complications.
Nodule / mass	A rounded opacity in the lung (nodule smaller than 3 cm,	Well- or ill-defined round opacity.	May be benign or malignant; requires further imaging and

Table 1.2: Thoracic abnormalities and their significance (Continued)

Abnormality	Description	Typical radiographic appearance	Clinical significance
	mass larger).		specialist review.

1.9 Methodology Followed

The project was developed in two phases. In Phase I (BCS685) the problem was analyzed, the literature was surveyed, the architecture and data-flow diagrams were designed and the diagnosis model was developed.

1.10 Applications and Benefits

1.10.1 Applications

- **Primary health centers and rural clinics:** an automatic first-level screening aid where no radiologist is available on site.
- **Diagnostic laboratories and imaging centers:** rapid triage of incoming radiographs and generation of a structured, printable certificate.
- **Hospital outpatient departments:** prioritization of patients with high-severity findings such as pleural effusion.

1.10.2 Benefits

Table 1.3: Benefits to each stakeholder

Stakeholder	Benefit
Patient	Quick, understandable result; visual marking of the abnormal region; direct suggestion of a suitable specialist; online appointment; all reports in one place.
Doctor	Pre-screened, prioritized list of consultation requests; images with localized findings and confidence values; one-click confirmation or custom prescription.
Laboratory	Faster turnaround; structured certificate; registry of processed tests; reduced manual paperwork.
Healthcare system	Better use of scarce specialists; earlier detection; a digital record that supports follow-up and audit.

1.11 Report Organization

Chapter 1 gives the overview, motivation, problem statement, scope, objectives, and a comparison of the existing and proposed systems.

Chapter 2 reviews the literature on AI-based pneumonia and chest X-ray diagnosis, interpretability methods, object detection and the datasets used, and identifies the research gaps addressed by this project.

Chapter 3 presents the system analysis, including feasibility studies and the functional and non-functional requirements.

Chapter 4 lists the hardware and software requirements and describes the technologies used.

Chapter 5 describes the system design: logical design, design goals, architecture, data flow diagrams, use-case model, workflow, database design and project schedule.

Chapter 6 describes the implementation of each module, including model training, inference, severity mapping, the doctor recommendation algorithm and report generation.

Chapter 7 presents the testing strategy, test cases and results.

Chapter 8 shows the screenshots and outputs of the working system.

Chapter 9 concludes the report and outlines the future scope.

1.13 Introduction Summary

This chapter introduced CliniScan+, an AI-powered chest X-ray diagnosis and healthcare support system. It explained why the project is needed, the problems in the current workflow, and the scope and objectives of the work, and briefly described the proposed system.

CHAPTER 2
LITERATURE SURVEY

A literature survey was carried out on AI-based diagnosis of pneumonia and other lung diseases from chest X-rays, with attention to model accuracy, interpretability, object detection and the datasets used. Six studies were reviewed in detail.

2.1 Enhancing Pneumonia Diagnosis Through AI Interpretability

NAME	YEAR	AUTHOR	ADVANTAGES	DISADVANTAGES
Enhancing Pneumonia Diagnosis Through AI Interpretability: Comparative Analysis of Pixel-Level Interpretability and Grad-CAM on X-ray Imaging with VGG19	2025	Mohammad Ennab, Hamid Mcheick	- Compares pixel-level interpretability (PLI) with Grad-CAM on three datasets. - PLI gives fine-grained explanations. - Evaluates time complexity, calibration and radiologist feedback.	- Compares only two XAI methods. - No hybrid interpretability or multi-model comparison. - Limited dataset diversity. - No real-time clinical deployment evaluation.

This study uses a VGG19 classifier and compares two explanation techniques for pneumonia detection:

- Pixel-Level Interpretability (PLI) produces detailed, fine-grained explanations of the pixels that influence the decision.
- Grad-CAM produces coarse heat-maps over the regions that influenced the prediction.
- The methods are assessed using computation time, calibration, standard performance metrics and feedback from radiologists.
- The study stresses that interpretability is essential for building clinical trust in AI.

The work confirms that explanations are necessary for clinical acceptance of AI. However,

it remains at the level of a classifier with post-hoc heat-maps and does not deploy the model in a clinical workflow.

2.2 AI-Driven Deep Learning Framework for Pneumonia Detection

NAME	YEAR	AUTHOR	ADVANTAGES	DISADVANTAGES
Revolutionizing Pneumonia Diagnosis: AI-Driven Deep Learning Framework for Automated Detection From Chest X-Rays (IEEE Access)	2024	N. Shilpa, W. Ayeesha, Prakash B. Metre	- Compares AlexNet, MobileNetV2, ResNet50, EfficientNetB0 and Xception. - CLAHE pre-processing and 5-fold cross-validation. - EfficientNetB0 reached 99.78% accuracy.	- Focuses only on classification accuracy. - No explainability evaluation. - No multi-class pneumonia severity grading. - Not linked to any clinical workflow.

This framework applies transfer learning to chest X-ray classification:

- Five pre-trained networks (AlexNet, MobileNetV2, ResNet50, EfficientNetB0 and Xception) are compared on the same data.
- Contrast-limited adaptive histogram equalization (CLAHE) improves the visibility of lung structures before training.
- Five-fold cross-validation gives a reliable estimate of performance.
- EfficientNetB0 performs best, with an accuracy of 99.78%.

The paper demonstrates the value of pre-processing and transfer learning, but it addresses only binary pneumonia classification and offers neither interpretability nor severity. CliniScan+ extends detection to ten pathologies, adds localization, severity and a clinical protocol, and connects the result to specialist referral.

2.3 Pneumonia Detection Using Deep Learning and Grad-CAM++

NAME	YEAR	AUTHOR	ADVANTAGES	DISADVANTAGES
Pneumonia Detection in Chest X-Ray Images Using Deep Learning and Grad-CAM++ Visualization (IEEE ASYU)	2024	Emre Şahin et al.	- Segmentation, pre-processing and pre-trained VGG19, EfficientNetB0 and Inception-V3. - EfficientNetB0 achieved about 95% accuracy. - Grad-CAM++ localized infected regions.	- Only Grad-CAM++ used; no comparison with other XAI methods. - Small dataset. - Segmentation not clinically validated. - No comparison with radiologist markings.

This system combines segmentation, classification and visual explanation:

- Lung segmentation and pre-processing prepare the radiographs for the classifiers.
- VGG19, EfficientNetB0 and Inception-V3 are trained by transfer learning; EfficientNetB0 gives about 95% accuracy.
- Grad-CAM++ produces heat-maps that highlight the infected regions and improve interpretability.

Heat-maps show where a classifier looked but do not give discrete, countable findings, and the evidence base is a small dataset without validation against radiologist annotations. Training a detector directly on radiologist-drawn boxes, as in CliniScan+, gives explicit and verifiable localization for each finding.

2.4 CheXNet: Radiologist-Level Pneumonia Detection

NAME	YEAR	AUTHOR	ADVANTAGES	DISADVANTAGES
CheXNet: Radiologist-Level Pneumonia Detection on Chest X-Rays with Deep Learning	2017	P. Rajpurkar et al.	- 121-layer DenseNet trained on a large public chest X-ray dataset. - Reported performance comparable to or above the average radiologist on a test set for pneumonia. - Class activation maps used to localise the pathology.	- Image-level classification of a single target (pneumonia). - Coarse heat-map localization only. - Training labels mined automatically from reports and may be noisy. - No workflow, referral or deployment component.

CheXNet showed that a deep network can reach expert-level performance on a defined chest X-ray task:

- A 121-layer DenseNet is trained on a large public dataset of frontal chest radiographs.
- The model outputs a probability of pneumonia and a class activation map that highlights the influential region.
- Its performance on a test set was compared with that of practicing radiologists.

The work inspired much later research and shows that deep learning is clinically promising. Its limitations for this project are that it addresses a single condition, provides only coarse localization and stops at prediction. CliniScan+ keeps the idea of clinically useful deep learning but moves to lesion-level, multi-pathology detection and adds the downstream workflow.

2.5 YOLOv11: Real-Time Object Detection

NAME	YEAR	AUTHOR	ADVANTAGES	DISADVANTAGES
YOLOv11: An Overview of the Key Architectural Enhancements	2024	R. Khanam, M. Hussain	- Single-pass detector that predicts boxes and classes together. - Improved backbone and neck (C3k2, SPPF, C2PSA attention) give better features with fewer parameters. - Nano to extra-large variants; the Nano model runs in real time on modest hardware.	- General-purpose model benchmarked on natural images, not medical images. - Small or low-contrast lesions are harder to detect. - Needs domain-specific training data, augmentation and threshold tuning.

This paper describes the architecture of the YOLO11 family of detectors:

- The backbone extracts multi-scale features using C3k2 blocks, a spatial pyramid pooling module and a C2PSA attention block.
- The neck fuses feature of different resolutions so that small and large objects are both represented.
- A decoupled head predicts the box and the class at three scales, and non-maximum suppression removes duplicate boxes.
- Several model sizes allow a trade-off between accuracy and speed.

Because the output of a detector (a box, a class and a score) can be read directly by a clinician, it is attractive for medical imaging. The Nano variant gives the lowest latency, which suits an interactive web application, and it is therefore the detector chosen for CliniScan+, after training on a chest X-ray dataset.

2.6 VinDr-CXR: A Chest X-ray Dataset with Radiologist Annotations

NAME	YEAR	AUTHOR	ADVANTAGES	DISADVANTAGES
VinDr-CXR: An Open Dataset of Chest X-rays with Radiologist's Annotations (basis of the VinBigData challenge)	2022	H. Q. Nguyen et al.	- Large set of chest radiographs annotated by experienced radiologists. - Bounding boxes (local labels) as well as image-level labels. - Well suited to training object detectors.	- Collected from a limited set of hospitals. - Class imbalance, with some abnormalities rare. - Inter-reader variation in annotations, especially for diffuse findings.

This dataset supplies the training data for lesion-level detection:

- Each radiograph is annotated by radiologists with rectangles around abnormalities and the abnormality class.
- Local (box-level) labels and global (image-level) labels are both provided.
- The data were used in the Vin Bigdata chest X-ray abnormalities detection challenge, which made them a common benchmark.

Datasets with only image-level labels support classification and heat-maps, whereas box-annotated data such as VinDr-CXR also support localization. CliniScan+ uses ten pathology classes from this data with a 70/15/15 train/validation/test split.

2.7 Literature Survey Summary

The reviewed literature shows that deep learning can classify pneumonia on chest X-rays with very high accuracy [2], [3], [9] and that interpretability is essential for clinical trust [1], [3]. Fast single-stage detectors such as YOLO11 [4] and box-annotated datasets such as VinDr-CXR [5] now make it possible to localize several abnormalities in a single pass. However, the existing studies mostly address binary pneumonia classification, explain their output only with heat-maps, do not grade severity, and do not connect the AI result to the next clinical step: the choice of a specialist, an appointment and a medical record.

CliniScan+ addresses these gaps by combining a YOLO11 Nano detector trained on Vin

Bigdata with bounding-box explainability, severity grading, a hybrid rule-based and MCDA doctor recommender, and a consultation tracking module in one platform, turning an AI prediction model into a complete digital healthcare support system.

CHAPTER 3
SYSTEM ANALYSIS

3.1 Introduction to System Analysis

A **system** is an orderly group of interdependent components linked together according to a plan to achieve a specific objective. **Analysis** is a detailed study of the operations performed by a system and their relationships within and outside the system.

3.2 Feasibility Study

Feasibility is the determination of whether a project is worth doing. The following types of feasibility were considered.

3.2.1 Technical Feasibility

The technologies required — Python, PyTorch, the Ultralytics YOLO framework, OpenCV, FastAPI, Next.js, MongoDB and the Google Gemini API — are mature and well documented, and all except the hosted Gemini service are open source. The YOLO11 Nano model is small enough to run inference on a standard laptop CPU, and a GPU is needed only for training.

3.2.2 Economic Feasibility

All software components are free and open source and the model can be trained on a consumer or free cloud GPU. No special medical hardware is needed because the system works on images that laboratories already produce.

3.2.3 Operational Feasibility

The three user groups — patients, doctors and laboratory staff — interact with the system through a browser with a simple, role-specific interface. No special skills are needed beyond basic computer use, and one-click demo logins help first-time users.

3.2.4 Ethical and Legal Considerations

Medical images and personal data are sensitive. The system uses authenticated, role-based access, stores data in a database rather than in the browser, and presents AI output as decision support that must be confirmed by a doctor.

3.2.5 Schedule Feasibility

The project had to be completed within the two semesters of the final year. The diagnosis model was developed in Phase I and the integration work in Phase II was planned as an eight-week schedule (Section 5.15).

3.3 Functional Requirements

Table 3.1: Functional requirements

ID	Requirement
FR1	The system shall allow patients, doctors and laboratories to register and sign in with role-based access.
FR2	The system shall accept a chest X-ray in JPEG or PNG format (DICOM-derived images) of up to 25 MB.
FR3	The system shall run the YOLO11 model and return every abnormality above a user-adjustable confidence threshold (5%–70%).
FR4	The system shall show bounding boxes, disease name, confidence and anatomical zone for each finding.
FR5	The system shall assign a severity level and a clinical protocol to each finding.
FR6	The system shall recommend a primary specialty and a ranked list of doctors from the doctor directory.
FR7	The patient shall be able to search the doctor directory and book an appointment.
FR8	The doctor shall be able to view pending requests, confirm an appointment in one click and add a custom prescription or advice.
FR9	The laboratory shall be able to register a patient, upload a radiograph and generate a diagnostic certificate.
FR10	The system shall store users, patients, X-ray records, AI reports, doctors, appointments and consultation history in a database.
FR11	The system shall provide a downloadable diagnostic report and a scan history for each patient.

3.4 Non-Functional Requirements

- **Performance:** the detection result should be displayed within about a second of

upload; the dashboard reports a median model inference latency of less than 250 ms.

- **Usability:** dashboards must be clear and consistent, with role-specific navigation and minimal steps to complete a task.
- **Security:** authentication, role-based access control and password protection must prevent unauthorized access to patient data.
- **Explainability:** every AI finding must be visible on the image and accompanied by a confidence value.

The non-functional requirements are made measurable in Table 3.1A so that they can be verified during testing.

Table 3.1A: Measurable non-functional requirements

Attribute	Measure	Target
Response time	Time from pressing Run to annotated image	About 1 s or less on the reference machine
Model latency	Median YOLO11 Nano inference time	Below 250 ms (dashboard figure)
Upload limit	Maximum image size	25 MB
Capacity	Doctors in directory / registered accounts	2,000+ doctors; 200+ accounts demonstrated
Availability	Operation if database service fails	Continues in in-memory fallback mode
Security	Unauthorized role access	Denied in 100% of tested cases
Usability	Tasks completed without help	At least 90% of test tasks
Portability	Operating systems and browsers	Windows, Linux, macOS; Chrome, Edge, Firefox, Safari

3.5 Software Requirement Specification

3.5.1 Purpose and Product Perspective

The purpose of this specification is to state precisely what CliniScan+ must do. The product is a stand-alone web application that consists of a browser front end, an application server

that hosts the inference engine and the recommender, and a document database.

3.5.2 Assumptions and Dependencies

- The user supplies a frontal (PA or AP) chest radiograph in JPEG or PNG format, or a JPEG/PNG derived from DICOM.
- The trained model weights and the doctor directory are available to the application server at start-up.
- A modern web browser and a network connection to the server are available to the user.
- The AI output is advisory; a qualified doctor takes the final clinical decision.

3.5.3 Constraints

- The upload size is limited to 25 MB per image.
- The model is limited to the ten pathology classes it was trained on; other findings will not be reported.
- The system is a prototype and must not be used as a certified medical device.
- Performance depends on the hardware of the server, especially the availability of a GPU.

3.5.4 External Interface Requirements

Table 3.2: External interfaces

Interface	Description
User interface	Responsive web pages with role-specific navigation, upload widgets, a confidence slider, image canvas with bounding boxes, cards for findings and doctors, and a printable certificate.
Hardware interface	Standard keyboard, mouse and display; no special devices are required.
Software interface	HTTP/JSON (REST) between the Next.js front end and the FastAPI back end; Python driver for MongoDB; Ultralytics/PyTorch API for the model; Google Gemini API for the Clinical Assistant.
Communication interface	HTTP(S) over a local or wide-area network; optional TLS on the server.

3.6 User Classes and Access Rights

CliniScan+ has three classes of user, each with a separate portal. The access rights are summarized in Table 3.3.

Table 3.3: Role-permission matrix

Function	Patient	Doctor	Laboratory
Register / sign in	Yes	Yes	Yes
Upload chest X-ray and run AI triage	Yes (own)	Yes (diagnostic studio)	Yes (for registered patients)
View findings, severity and protocol	Yes (own)	Yes	Yes
Receive specialist recommendation	Yes	View	Yes (routing)
Search doctor directory	Yes	Yes	Yes
Book appointment	Yes	No	No
Confirm appointment / add prescription	No	Yes	No
View own scan history and certificates	Yes	For confirmed patients	Processed tests registry
Edit profile / facility details	Yes	Yes (practice settings)	Yes (facility profile and licence)

3.7 Use Case Specifications

The main use cases are specified in the following tables.

Table 3.4: UC-1 Register and sign in

Item	Description
Actor	Patient, doctor, laboratory
Pre-condition	The user has a browser and access to the portal.
Main flow	1. User opens the landing page. 2. Chooses Register or Sign in. 3. Selects role and enters details. 4. System validates and creates or authenticates the account. 5. The role-specific portal is opened.

Table 3.4: UC-1 Register and sign in (Continued)

Item	Description
Alternate flow	Invalid credentials: an error message is shown and the user can retry. Duplicate username: registration is rejected.
Post-condition	A session is established with the role of the user.

Table 3.5: UC-2 Run AI triage on a chest X-ray

Item	Description
Actor	Patient or laboratory (doctor through the diagnostic studio)
Pre-condition	The user is signed in.
Main flow	1. User uploads a radiograph or selects a sample. 2. Sets the confidence threshold. 3. Runs the triage. 4. System validates and pre-processes the image. 5. YOLO11 returns detections. 6. System maps zones, severity and protocol. 7. Annotated image and findings are displayed.
Alternate flow	Unsupported or corrupt file: the file is rejected. No detection above the threshold: the system reports no significant finding.
Post-condition	The X-ray record and AI report are stored.

Table 3.6: UC-3 Obtain specialist recommendation

Item	Description
Actor	Patient / laboratory
Pre-condition	AI report exists with at least one finding.
Main flow	1. System applies clinical triage rules to the findings. 2. Selects the primary specialty and urgency protocol. 3. Retrieves doctors of that specialty. 4. Computes the MCDA score for each. 5. Displays the top-ranked doctors with rating, hospital, experience and fee.
Alternate flow	No doctor available for the specialty: a general physician is suggested.
Post-condition	A ranked list is shown and available for booking.

Table 3.7: UC-4 Book and confirm an appointment

Item	Description
Actor	Patient, doctor
Pre-condition	A recommendation or directory listing is displayed.
Main flow	1. Patient selects Schedule Consultation. 2. System creates an appointment with status Pending. 3. Doctor opens the review queue. 4. Doctor confirms in one click or enters a custom prescription and advice. 5. Status changes to Confirmed. 6. Patient sees the confirmation and can download the referral pass.
Alternate flow	Doctor declines or the slot is unavailable: the status is updated and the patient may choose another doctor.
Post-condition	Appointment and consultation history are stored.

Table 3.8: UC-5 Laboratory intake and certificate

Item	Description
Actor	Laboratory
Pre-condition	Laboratory is signed in.
Main flow	1. Laboratory enters patient name, MRN, age, gender, city and referring physician. 2. Uploads the radiograph. 3. Runs automated triage. 4. System generates findings and a diagnostic certificate. 5. Entry is added to the laboratory registry.
Alternate flow	Required fields missing: the form is not submitted.
Post-condition	Registry contains the processed test.

3.8 Requirements Traceability Matrix

Table 3.9: Traceability of requirements

Requirement	Design element	Implementation module	Verified by
FR1 Authentication and roles	Users collection; role check	Authentication module	TC1–TC3, ST1–ST3

Table 3.9: Traceability of requirements (Continued)

Requirement	Design element	Implementation module	Verified by
FR2 Image upload	Upload form; size/type validation	Data acquisition module	TC4, TC5, UT1
FR3 Detection with threshold	YOLO11 inference; confidence slider	Detector module	TC6, TC7, UT3
FR4 Boxes, class, confidence, zone	Explainability module	Renderer and zone mapper	TC6, UT4
FR5 Severity and protocol	Rule table	Severity module	TC8, UT5
FR6 Specialist recommendation	Triage rules; MCDA	Recommender module	TC9, UT6, UT7
FR7–FR8 Appointments	Appointments collection	Consultation module	TC10, TC11, IT4–IT6
FR9 Laboratory workflow	Lab portal	Laboratory module	TC12
FR10 Persistence	Seven collections	Database layer	UT8, IT1
FR11 Reports and history	Certificate template	Report generator	TC13, TC14

3.9 Risk Analysis

Table 3.10: Project risks and mitigation

Risk	Likelihood	Impact	Mitigation
Model misses a lesion (false negative)	Medium	High	Adjustable low threshold; doctor confirmation; clear advisory disclaimer.
False alarm (false positive)	Medium	Medium	Display confidence; severity grading; doctor review before decisions.
Class imbalance in the dataset	High	Medium	Augmentation; class-weighted evaluation; per-class reporting.

Table 3.10: Project risks and mitigation (Continued)

Risk	Likelihood	Impact	Mitigation
Database unavailable	Low	High	In-memory fallback store; error handling.
Unauthorized access to patient data	Low	High	Authentication, role-based access, hashed passwords.
Slow inference on weak hardware	Medium	Medium	Nano model; optional GPU; image size limit.
Schedule slip	Medium	Medium	Incremental development; eight-week plan with weekly review.

3.10 System Analysis Summary

This chapter established that the project is technically, economically and operationally feasible and listed the functional and non-functional requirements that guide the design and testing of the system.

CHAPTER 4
REQUIREMENT ANALYSIS

System requirements are the configuration that a system must have in order for a hardware or software application to run smoothly and efficiently. Failure to meet them can cause installation or performance problems. The requirements of CliniScan+ are grouped into hardware and software requirements.

4.1 Hardware Requirements

Table 4.1: Hardware requirements

Component	Specification
Processor	Intel Core i5 / AMD Ryzen 5 or higher
RAM	8 GB minimum (16 GB recommended)
Storage	256 GB SSD
GPU	Optional NVIDIA GPU (recommended for model training)
Input devices	Keyboard and mouse
Output device	Monitor

4.2 Software Requirements

Table 4.2: Software requirements

Component	Specification
Operating system	Windows / Linux / macOS
Programming language	Python
Deep learning framework	PyTorch with Ultralytics YOLO11 Nano
Libraries	OpenCV, NumPy, Pandas
Front end	Next.js 14 (React 18, TypeScript, Tailwind CSS, Lucide-React)

Table 4.2 : Software requirements (Continued)

Component	Specification
Back end	Fast API (v0.110+) running on ASGI Uvicorn server
Database	MongoDB with seamless in-memory fallback (mongomock) and local JSON audit backup, ensuring zero downtime if the MongoDB service is offline
Explainability	Bounding-box localization with confidence scores and anatomical quadrant mapping
LLM / Clinical Decision Support	Google Gemini API (Gemini-powered Clinical Assistant)
Tools	VS Code, Jupyter Notebook

4.3 Description of Technologies

4.3.1 Python

Python is the primary language of the project. Its rich ecosystem for machine learning, image processing and web development allows the model, the recommender and the web back end to be written in one language.

4.3.2 PyTorch and Ultralytics YOLO11

PyTorch provides tensor computation and automatic differentiation for training. The Ultralytics library provides the YOLO11 architecture, training pipeline, data augmentation and inference API. The Nano variant was selected because of its small size and low latency.

4.3.3 OpenCV, NumPy and Pandas

OpenCV is used to read images, resize and normalise them and to draw bounding boxes and labels. NumPy handles array operations and Pandas is used to manage the doctor directory and to prepare dataset annotations.

4.3.4 FastAPI and Uvicorn

FastAPI (v0.110+) provides the back-end REST engine. It receives multipart uploads, calls the YOLO11 model and the recommender, invokes the Gemini Clinical Assistant, and reads

and writes the database. It runs on the ASGI Uvicorn server (port 8000), which handles requests asynchronously and generates interactive API documentation automatically. The early prototype interface was replaced by the production Next.js front end, which offers the same functionality as a polished role-based dashboard.

4.3.5 MongoDB

MongoDB is a document database that stores flexible JSON-like documents. It suits medical records whose structure varies between patients (for example, a variable number

of findings per scan). The system uses seven collections. If the database service is unavailable, the data layer switches seamlessly to an in-memory store (mongomock) and keeps a local JSON audit backup, so the application continues to run with zero downtime.

4.3.6 VS Code and Jupyter

VS Code was used for application development and Jupyter Notebook for data exploration, training experiments and evaluation of the model.

4.3.7 Next.js, React and TypeScript

The user interface is built with Next.js 14 (React 18) and TypeScript, styled with Tailwind CSS and Lucide-React icons, and runs on port 3000. Reusable components such as XrayScanner and ClinicalChatbot handle image upload, the confidence slider, display of annotated findings and the chat panel, and call the FastAPI REST engine over HTTP/JSON. Component-based rendering and static typing keep the portals responsive, maintainable and consistent across the patient, doctor and laboratory roles.

4.3.8 Authentication and Password Security

The back end verifies credentials against salted password hashes, so plain-text passwords are never stored, and issues a signed session credential that carries the user identifier and role. Requests with invalid credentials are rejected with an HTTP 401 response and an informative error message.

4.3.9 Google Gemini API

The Google Gemini API provides the large language model behind the Clinical Assistant. The FastAPI service sends the patient's message, together with the findings of the current scan, to Gemini under a fixed system instruction, and returns the generated reply to the

Next.js chat panel. The model is used only for explanation and guidance, not for diagnosis.

4.3.10 Git and Version Control

The source code, configuration and documentation were kept under Git version control. Each module was developed on a branch and merged after testing, so that earlier working versions could always be restored.

4.4 Dataset Description

The detector is trained on the Vin Bigdata chest X-ray abnormalities dataset, derived from the VinDr-CXR collection [5]. Each image is a frontal chest radiograph and each annotation is a rectangle drawn by a radiologist around an abnormality, together with the class of that abnormality. Several radiologists may annotate the same image, and images without any abnormality carry a “no finding” label.

Table 4.3: Dataset annotation format

Field	Meaning
image_id	Unique identifier of the radiograph
class_name / class_id	Name and numeric id of the abnormality
rad_id	Identifier of the annotating radiologist
x_min, y_min, x_max, y_max	Corner coordinates of the bounding box in pixels

For this project ten abnormality classes were selected . Annotations were converted to the YOLO format, in which every line of a label file contains the class index and the normalized center coordinates, width and height of one box. The images were divided at random into training (70%), validation (15%) and test (15%) sets.

Table 4.4: Data split

Subset	Share	Purpose
Training	70%	Learning the network weights
Validation	15%	Model selection, threshold tuning, early stopping
Test	15%	Final unbiased evaluation

4.5 Theoretical Background

4.5.1 Convolutional Neural Networks

A convolutional neural network (CNN) is a deep network that applies learnable filters across an image. Early layers respond to edges and textures, and deeper layers combine them into shapes and structures. Weight sharing makes CNNs efficient for images, and pooling or stride convolution reduces resolution while increasing the receptive field.

4.5.2 Object Detection

An object detector solves two problems together: localization (where is each object?) and classification (what is it?). For every detected object it outputs a rectangular bounding box, a class label and a confidence score.

4.5.3 YOLO11 Architecture

YOLO11 is a member of the YOLO (You Only Look Once) family. It consists of a backbone, a neck and a detection head, as shown in Figure 4.1. The **backbone** extracts feature at several scales using convolutional stems, C3k2 blocks, a spatial pyramid pooling module (SPPF) and a C2PSA attention block. The **neck** fuses feature maps of different resolutions by up-sampling and concatenation so that both small and large lesions are represented. The **head** is decoupled: separate branches predict the box and the class at three scales (P3, P4, P5). The Nano variant uses the smallest depth and width multipliers, giving a model with a few million parameters that runs in real time on modest hardware [4], [7].

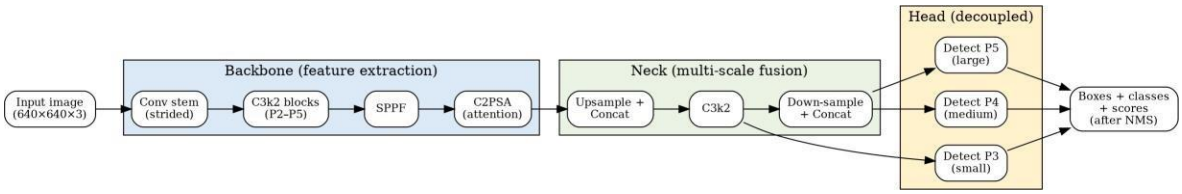


Fig 4.1: Block diagram of the YOLO11 detector

4.5.4 Intersection over Union and Non-Maximum Suppression

The overlap between a predicted box and a ground-truth box is measured by Intersection over Union (IoU), the area of their intersection divided by the area of their union. A prediction is counted as correct if its IoU with a ground-truth box of the same class exceeds a threshold (0.5 for mAP@0.5).

```
IoU(A, B) = Area(A ∩ B) / Area(A ∪ B)
```

4.5.5 Confidence Threshold

Each box has a confidence score between 0 and 1. Only boxes with a score above the threshold are shown. A low threshold (5%–15%) increases sensitivity, so fewer lesions are missed, but more false positives are shown.

4.5.6 Evaluation Metrics

The following standard metrics are used to evaluate the detector. TP, FP and FN denote true positives, false positives and false negatives at the chosen IoU threshold.

```
Precision  P = TP / (TP + FP)
Recall     R = TP / (TP + FN)
F1 score   F1 = 2 · P · R / (P + R)
AP (class) = area under the precision–recall curve for that class
mAP@0.5    = mean of AP over all classes at IoU = 0.5
mAP@0.5:0.95 = mean of mAP over IoU thresholds 0.5, 0.55, ..., 0.95
```

Precision tells how many reported findings are real; recall tells how many real findings were found. Average precision summarizes the trade-off between the two for a class and the mean over classes gives mAP.

4.5.7 Weighted MCDA Ranking

Each doctor is described by criteria of different units (rating out of 5, years of experience, fee in rupees, distance). Every criterion is first normalised to the range 0–1, using min–max normalisation for benefit criteria (higher is better) and its complement for cost criteria (lower is better, such as fee).

```
x_norm = (x - x_min) / (x_max - x_min) x_norm = 1 - (x - x_min) / (x_max -
x_min) Score(d) = Σ_c  w_c · x_norm,c(d)  with
   # benefit criteria
   # cost criteria Σ w_c = 1
```

4.6 Development and Runtime Environment

Table 4.5: Development environment

Item	Details
Python version	3.10.11
Ultralytics / PyTorch versions	Ultralytics 8.4.174

Table 4.5: Development environment(Continued)

Item	Details
OpenCV / NumPy / Pandas versions	OpenCV (cv2) 5.0.0/NumPy 2.2.6/Pandas 2.3.3
FastAPI / Uvicorn / PyMongo versions	FastAPI 0.142.2 with Uvicorn 0.54.0 / PyMongo 4.18.2
Front end framework	Next.js 14 (React 18, TypeScript, Tailwind CSS, Lucide-React)
LLM / Clinical Decision Support	Google Gemini API
MongoDB version	MongoDB 6.0 / 7.0+
Training platform	Local GPU
Supported browsers	Current versions of Chrome, Edge, Firefox and Safari
Network	HTTP/HTTPS over LAN or internet

Running the application requires installing the Python dependencies and the Node.js packages of the front end, placing the trained weights in the model folder, configuring the Google Gemini API key, starting the MongoDB service (or allowing the in-memory fallback) and launching the FastAPI server (port 8000) and the Next.js front end (port 3000). The user then opens the portal in a browser.

4.7 Requirement Analysis Summary

This chapter listed the hardware and software required to build and run CliniScan+ and described the role of each technology.

CHAPTER 5
SYSTEM DESIGN

System design defines the architecture, components, modules, interfaces and data of a system so that it satisfies the specified requirements. This chapter describes the design of CliniScan+ using architecture, data flow, use-case, workflow and database diagrams.

5.1 Logical Design

The logical design of CliniScan+ divides the system into five cooperating modules that communicate through the web application and share a common database:

- **AI Diagnosis Module:** receives the X-ray, pre-processes it and runs YOLO11 Nano to obtain detections.
- **Explainable AI Module:** draws bounding boxes, labels and confidence scores on the radiograph and maps each box to an anatomical zone.
- **Results Aggregation Module:** produces the disease list, confidence scores, severity levels and clinical protocols.
- **Doctor Recommendation Module:** selects the specialty using rule-based triage and ranks doctors with weighted MCDA.
- **Gemini Clinical Assistant Module: answers patient questions with Google Gemini, explains findings, detects emergency red flags and guides consultations.**
- **Consultation Tracking and Appointment Module:** books and confirms appointments, and records prescriptions, follow-up and history.

5.2 Design Goals

- **Accuracy and speed:** near-real-time detection using a lightweight detector.
- **Explainability:** every prediction must be visible and verifiable by the doctor.
- **Actionability:** the output must lead to a specific next step (severity, protocol, specialist, appointment).
- **Usability:** role-specific dashboards with minimal steps.
- **Modularity and scalability:** replaceable modules and a database that scales to thousands of records.

- **Security:** authenticated, role-based access to sensitive patient data.

5.3 System Architecture

The architecture of CliniScan+ is a layered, integrated multi-module system. Users (patient, diagnostic laboratory and doctor) interact with a web front end that provides authentication, role management, dashboard and navigation.

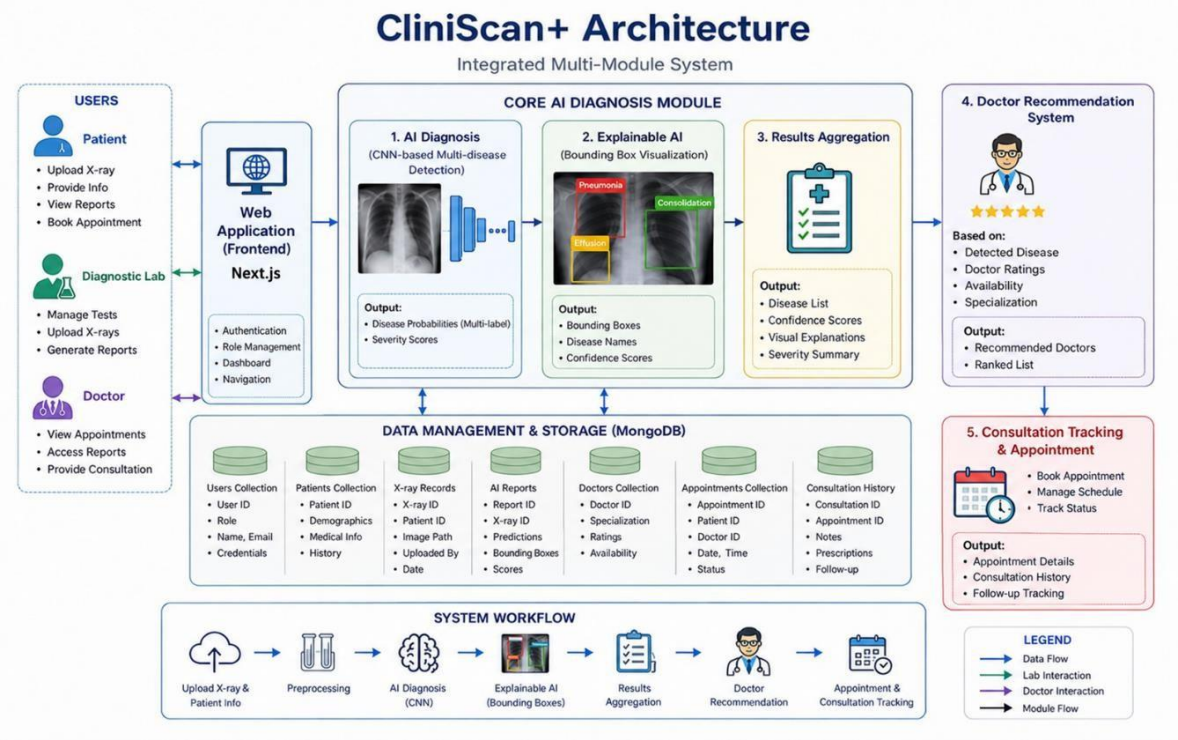


Fig 5.1: System architecture of CliniScan+

User Interface

The web interface is built with Next.js 14 (React 18, TypeScript and Tailwind CSS) and provides three role-specific portals. The patient portal offers X-ray scanning and specialist matching, the doctor directory, appointments and referrals, the diagnostic scan vault, the profile and the Gemini-powered Clinical Assistant.

AI Diagnosis Module

The uploaded image is validated, resized and normalized, and passed to the YOLO11 Nano network. The network outputs candidate boxes with class probabilities. Non-maximum suppression removes overlapping duplicates and the user-selected confidence threshold (5% to 70%) filters low-confidence detections.

Doctor Recommendation Module

Detected pathologies are mapped by clinical rules to a primary specialty (for example, ILD,

pleural effusion and pleural thickening map to Pulmonology). Doctors of that specialty are then scored and ranked by a weighted multi-criteria formula.

Gemini AI Clinical Assistant

The Gemini AI Clinical Assistant is a conversational module that is available 24 hours a day from the patient portal. It is powered by the Google Gemini large language model, which the FastAPI REST engine calls through a dedicated client (gemini_chatbot.py). Each request carries the patient's question together with the structured findings of the current scan (pathology, location, confidence and severity), so that answers refer to the actual result rather than to generic information. The assistant is a decision-support aid: it is instructed never to give a final diagnosis and to present AI findings as information that a qualified doctor must confirm.

Explaining radiologist findings. The assistant translates radiologist-style findings such as interstitial lung disease or pleural effusion into clear, empathetic language, explains what the severity label and recommended protocol mean, and answers follow-up questions in a calm tone that does not alarm the patient.

Red-flag detection. Every message is screened for symptoms that suggest an acute thoracic emergency, such as severe breathlessness, chest pain, coughing blood, bluish lips or sudden collapse. When a red flag is detected, the assistant gives immediate emergency-room triage advice before any other content.

Consultation navigation. The assistant helps patients choose the right specialty, understand the ranked list produced by the MCDA recommendation engine, and find and book a specialist consultation among the doctors available across 26 cities.

Data Management and Storage

MongoDB holds the Users, Patients, X-ray Records, AI Reports, Doctors, Appointments and Consultation History collections. If the database service is not reachable, the data layer switches seamlessly to an in-memory engine (mongomock) and writes a local JSON audit backup, so the application keeps running with zero downtime.

Security

Access is limited by role after authentication. Passwords are not stored in plain text, and each portal exposes only the functions relevant to that role.

5.4 Data Flow Diagrams

5.4.1 Level 0 DFD (Context Diagram)

At Level 0 the whole system is one process. The patient supplies a chest X-ray image and receives a diagnosis with suggestions; a patient report is passed to the doctor.

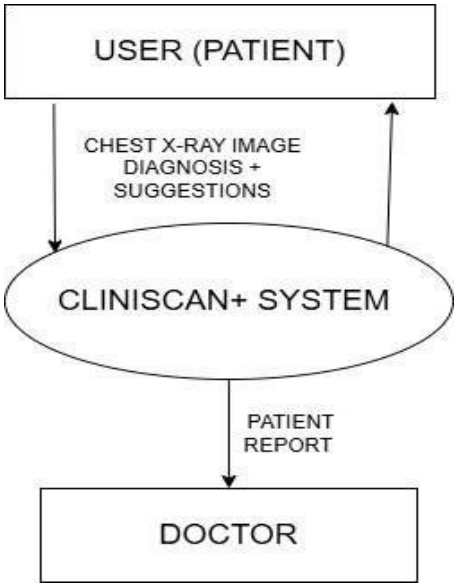


Fig 5.2: Level 0 DFD

5.4.2 Level 1 DFD

At Level 1 the process is decomposed into image upload, pre-processing, AI detection, explainable AI and recommendation. Detection results are stored in the patient database, the recommendation is returned to the patient as the diagnosis, and the report is saved in the medical records store.

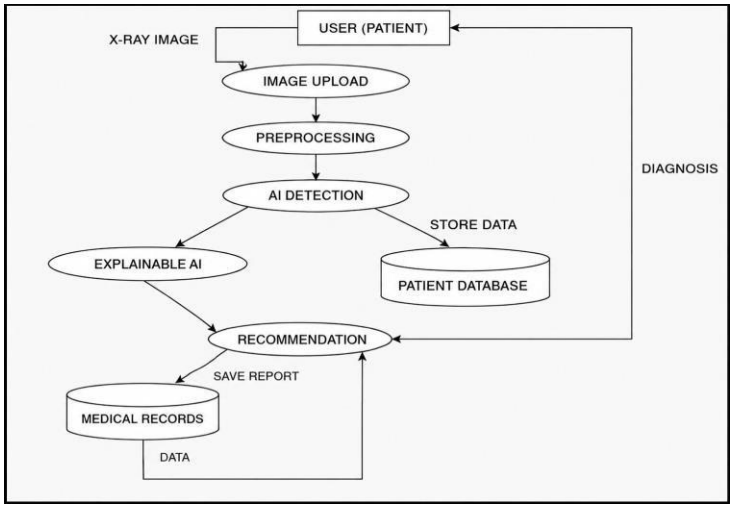


Fig 5.3: Level 1 DFD

5.5 Use Case Diagram

Three actors interact with the system. The patient uploads X-rays, views findings and recommendations, books appointments and downloads certificates. The doctor reviews and confirms appointment requests and adds prescriptions. The diagnostic laboratory registers patients, runs scans and generates certificates.

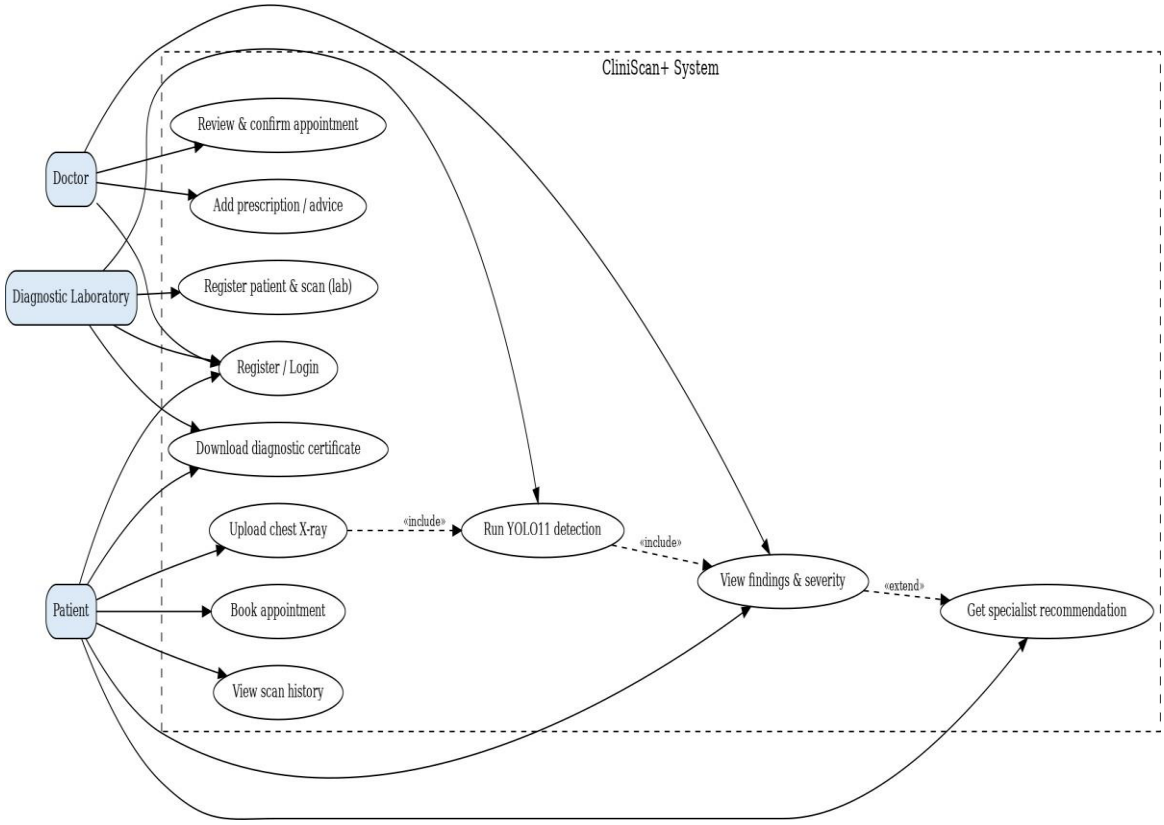


Fig 5.4: Use case diagram

5.6 Workflow (Activity) Diagram

Figure 5.5 shows the end-to-end workflow from login to stored report.

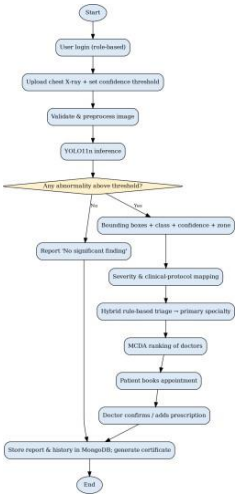


Fig 5.5: Workflow of CliniScan+

5.7 Database Design

The database contains seven collections. Their relationships are shown in Figure 5.6 and their main fields in Table 5.1.

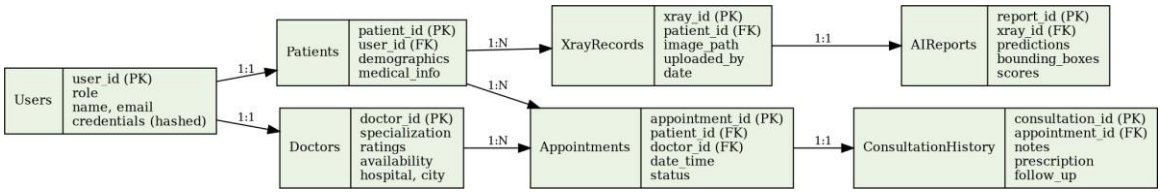


Fig 5.6: Entity relationship diagram

Table 5.1: Database collections

Collection	Main fields
Users	User ID, role, name, email, credentials
Patients	Patient ID, demographics, medical information, history
X-ray Records	X-ray ID, patient ID, image path, uploaded by, date
AI Reports	Report ID, X-ray ID, predictions, bounding boxes, scores
Doctors	Doctor ID, specialization, ratings, availability, hospital, city, experience, fee
Appointments	Appointment ID, patient ID, doctor ID, date and time, status
Consultation History	Consultation ID, appointment ID, notes, prescriptions, follow-up

5.8 Class Diagram

The object-oriented structure of the system is shown in Figure 5.7. The abstract User class is specialized into Patient, Doctor and Laboratory. Patients and laboratories upload X-ray records, which are analysed by the Detector to produce an AI Report.

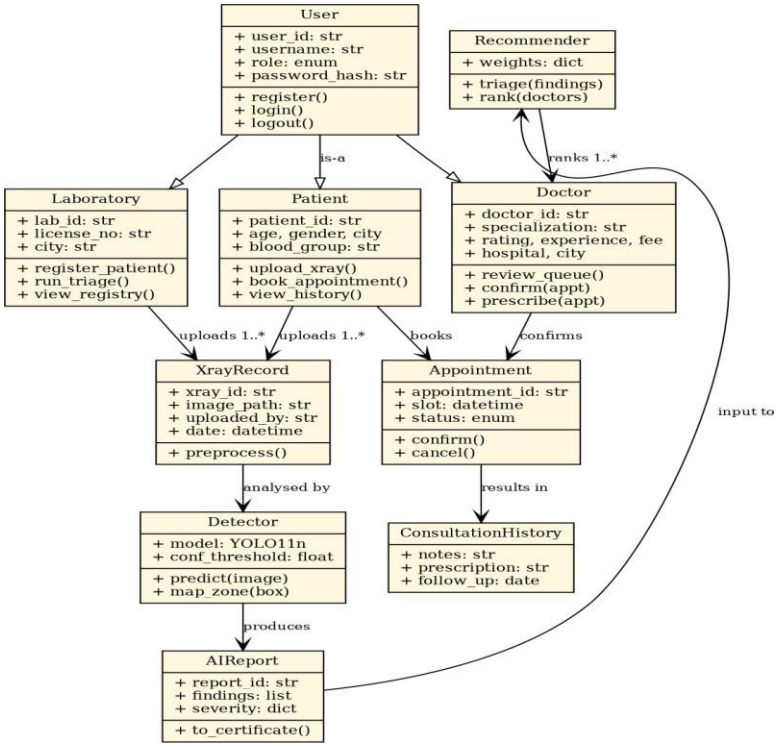


Fig 5.7: Class diagram of CliniScan+

5.9 Sequence Diagrams

5.9.1 AI Triage and Recommendation

Figure 5.8 shows the messages exchanged when a patient signs in, uploads a radiograph and receives findings with recommended doctors. The FastAPI REST engine validates and pre-processes the image, calls the detector, maps zones, severity and protocol, asks the recommender for a ranked list, stores the result in MongoDB and returns everything to the interface.

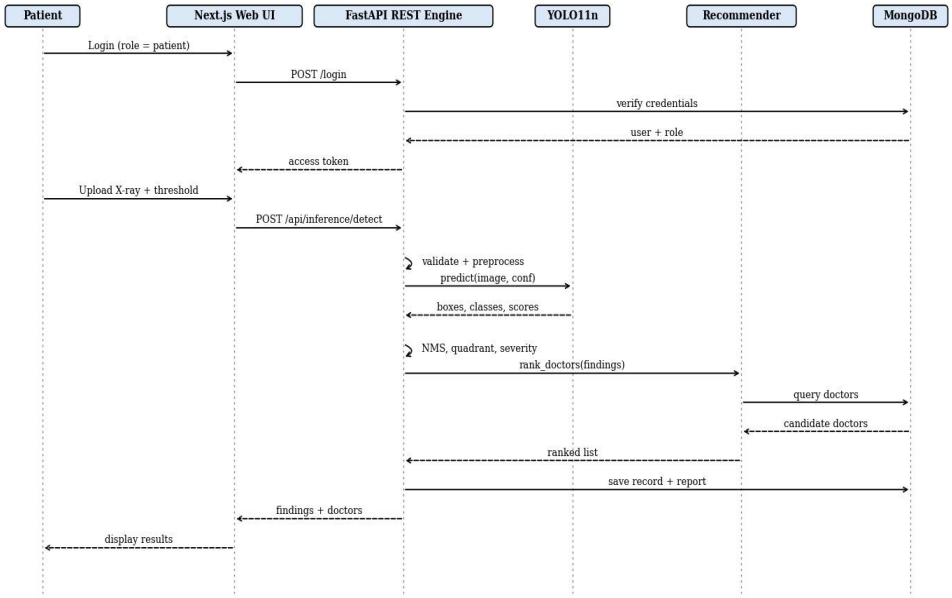


Fig 5.8: Sequence diagram of the AI triage flow

5.9.2 Appointment Booking and Confirmation

Figure 5.9 shows how an appointment is created with pending status, how the doctor reviews the queue and confirms, and how the patient later sees the confirmed status and referral pass.

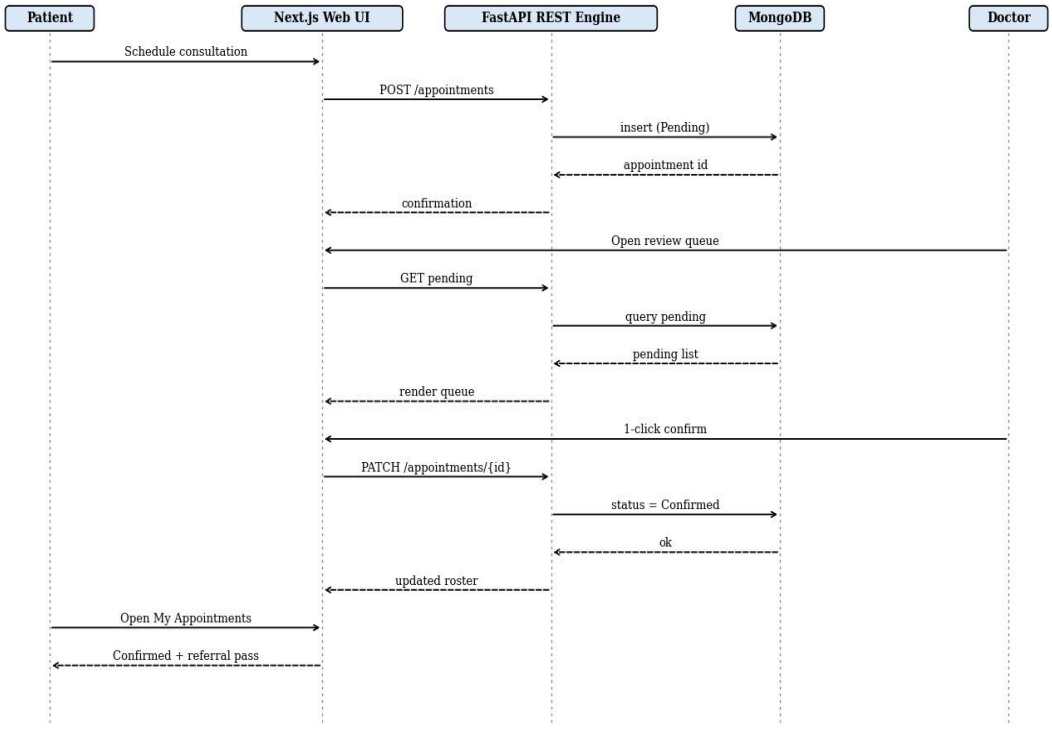


Fig 5.9: Sequence diagram of appointment booking and confirmation

5.10 State Diagram of an Appointment

An appointment passes through the states shown in Figure 5.10. It starts as a request,

becomes Pending Doctor Confirmation after the patient books a slot, and is then either confirmed or cancelled. A confirmed appointment ends as a completed consultation when notes and prescription are saved.

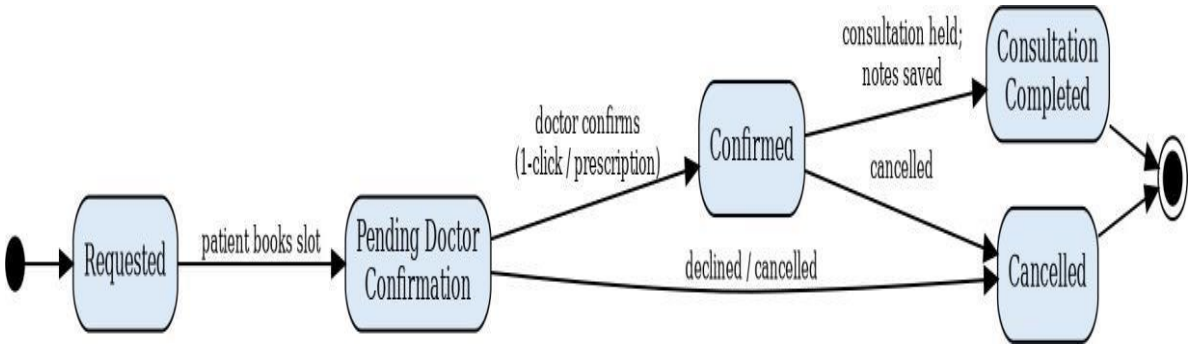


Fig 5.10: State diagram of an appointment

5.11 Component and Deployment View

The system is deployed in three tiers (Figure 5.11). The client tier consists of browsers running the three portals. The application tier hosts the Next.js 14 front end (port 3000) and the FastAPI service (Uvicorn, port 8000), which contains the YOLO11 inference engine, the recommender, the Gemini Clinical Assistant client and the report generator; the two are connected through REST/JSON. The Clinical Assistant calls the external Google Gemini API over HTTPS. The data tier consists of MongoDB, the in-memory fallback store with its JSON audit backup, and image storage.

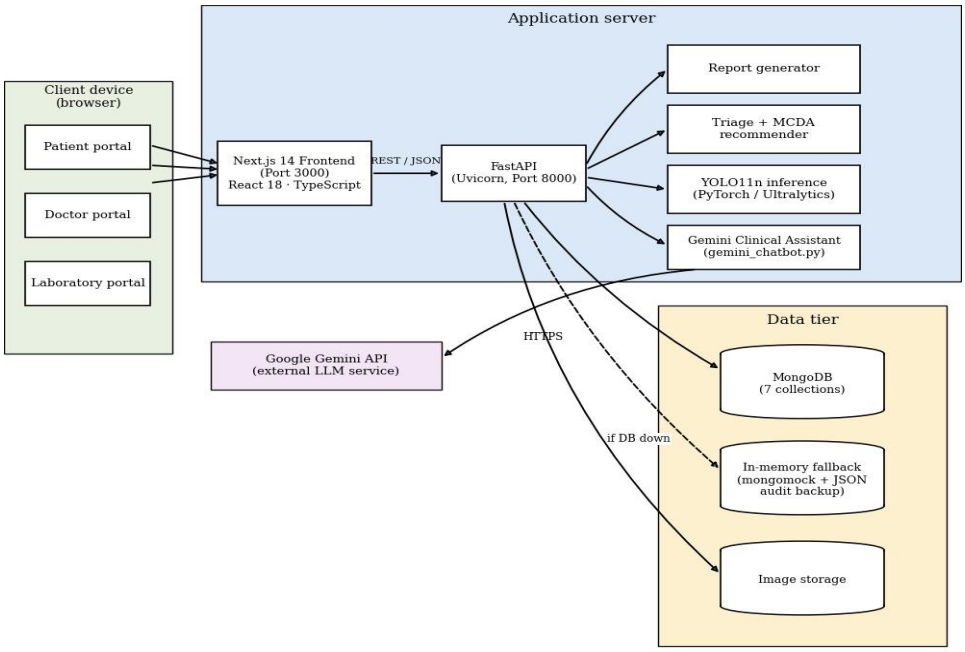


Fig 5.11: Deployment view of CliniScan+

5.12 Level 2 Data Flow Diagram: AI Detection

The AI detection process of the Level 1 DFD is decomposed in Figure 5.12 into validation, pre-processing, inference with NMS, zone mapping and severity lookup, box rendering and triage with MCDA ranking. The data stores used are X-ray records, AI reports and the doctor directory.

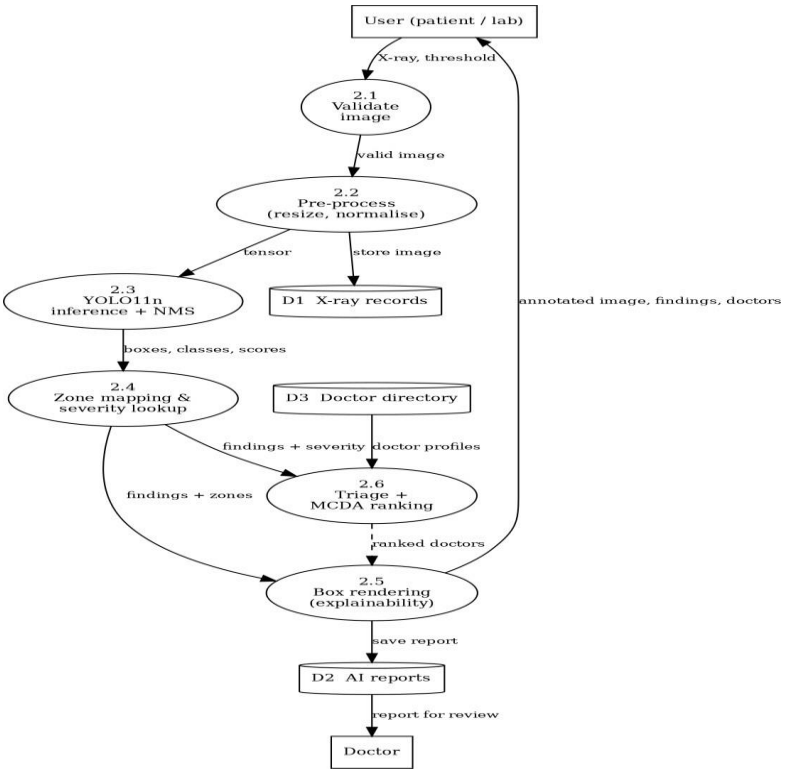


Fig 5.12: Level 2 DFD of the AI detection process

5.13 User Interface Design

The interface follows a consistent layout: a header with the product name, the signed-in user, a role switch and sign-out; a navigation hub on the left listing the functions of the current role; a hero banner giving the purpose of the page; and content cards on the right. A side panel called Clinical Infrastructure Specs always shows the detection engine (YOLO11 Nano), persistence mode, specialist roster size, number of pathologies and median latency so that the user knows what is running.

Table 5.2: Screens of the application

Screen	Role	Purpose	Main elements
Landing / Clinical Gateway	All	Sign in or register; one-click demo logins	Role selector, credentials form, demo accounts, platform statistics
Scan X-ray and Match Doctor	Patient	Upload and analyse an X-ray	Upload / sample selector, confidence slider, radiograph canvas, findings cards, recommended specialists
Doctors Directory	All	Search the 2,000+ doctors	Search filters, doctor cards with rating, hospital, experience, fee
My Appointments and Referrals	Patient	Track bookings	Status badges, prescription notes, referral pass download
Diagnostic Scan Vault	Patient	Archive of scans	Scan list with findings, report and certificate links
Patient Triage Review Queue	Doctor	Review and confirm requests	Pending and confirmed counters, 1-click confirm, custom prescription, roster
Diagnostic Studio Lab	Doctor	Run AI on an X-ray	Same analysis tools as the patient portal
Laboratory Intake and AI Triage	Laboratory	Register patient and run AI	Patient form, upload, certificate panel
Test Registry / Facility Profile	Laboratory	History and accreditation	Processed tests list; licence, contact and city settings
Model and Research Insights	All	Information about the model	Model, dataset and performance description

5.14 Security Design

- **Authentication:** every page other than the landing page requires a signed-in session.
- **Authorization:** the role stored in the session decides which navigation items and API functions are available (Table 3.3).
- **Credential protection:** passwords are stored as salted hashes, never in plain text.

- **Input validation:** uploaded files are checked for type and size (25 MB); text inputs are sanitized before storage and before display in the certificate.
- **Data minimization:** the certificate and records contain only the information needed for care.
- **Transport security:** the deployment should be served over HTTPS.
- **Audit:** appointments and consultation notes carry timestamps and the identity of the confirming doctor.

5.15 Project Schedule (Gantt Chart)

Because the CNN-based diagnosis model and the clinical model were completed earlier, the second phase concentrated on system integration, front-end development, database design, explainability visualization, doctor recommendation, appointment scheduling, testing and deployment over an eight-week plan.

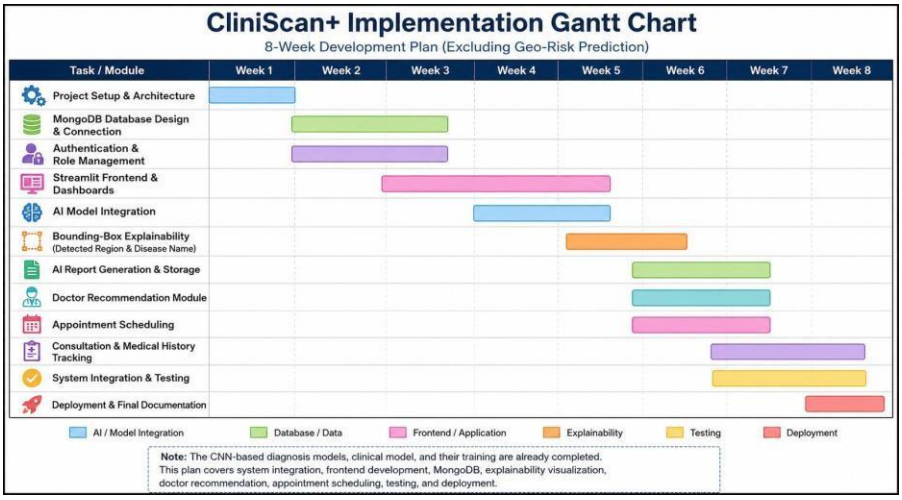


Fig 5.13: Implementation Gantt chart

Table 5.3: Task schedule

Task / module	Weeks
Project setup and architecture	1
MongoDB database design and connection	2–3
Authentication and role management	2–3
Front end and dashboards	3–5
AI model integration	4–5
Bounding-box explainability	5–6

Table 5.3: Task schedule (Continued)

Task/ module	Weeks
AI report generation and storage	6–7
Doctor recommendation module	6–7
Appointment scheduling	6–7
Consultation and medical history tracking	7–8
System integration and testing	7–8
Deployment and final documentation	8

5.16 Module Interface Specification

Table 5.4 specifies the input, processing and output of each module, which defines the interfaces between them.

Table 5.4: Module interface specification

Module	Input	Processing	Output
Authentication	Username, password, role	Hash comparison; session creation	Session with user id and role
Data acquisition	Image file, confidence threshold, city	Type and size check; decoding; optional CLAHE	Image array; threshold
Detector	Image array, threshold	YOLO11 inference; NMS; threshold filter	Boxes, classes, confidences, inference time
Explainability renderer	Image, boxes, labels	Draw boxes and labels; zone mapping	Annotated image; zone for each finding

Table 5.4: Module interface specification (Continued)

Module	Input	Processing	Output
Severity module	Finding label	Rule-table lookup	Severity, clinical protocol
Triage engine	Graded findings	Specialty voting; urgency from worst finding	Primary specialty, urgency protocol
MCDA ranker	Doctors of specialty, patient city	Normalization; weighted sum; sort	Top-N doctors with scores
Appointment module	Patient, doctor, slot, indication	Create and update appointment records	Appointment with status
Consultation module	Finding label	Rule-table lookup	Severity, clinical protocol
Triage engine	Appointment, notes, prescription	Store notes and follow-up	Consultation history entry
Report generator	Patient data, AI report, annotated image	Template rendering with escaping	Printable HTML certificate
Persistence layer	Documents from all modules	Insert, query, update; fallback on failure	Stored or retrieved documents

5.17 Recommendation Algorithm Flow

Figure 5.14 summarizes the logic of the recommendation module. The triage stage decides the specialty and urgency; the ranking stage orders the doctors of that specialty.

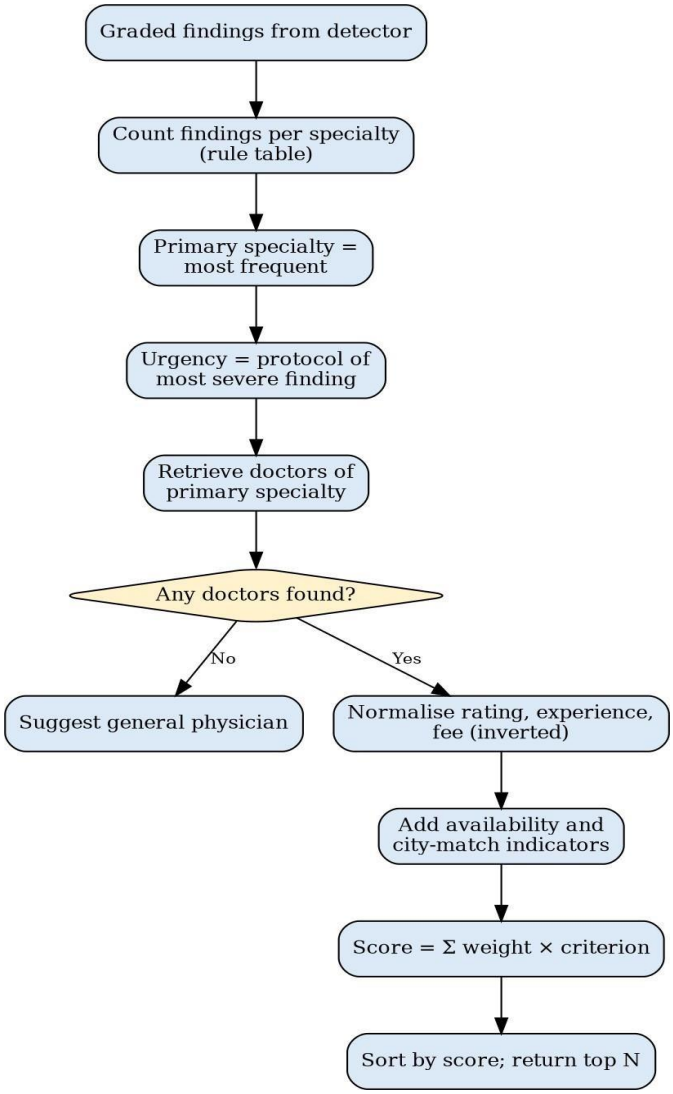


Fig 5.14: Flow of the hybrid triage and MCDA recommendation

Because the rules and weights are stored as data rather than hard-coded in the control flow, they can be reviewed and tuned by clinicians without changing the program logic.

5.18 Input and Output Design

Table 5.6: Main input forms

Form	Fields	Validation
Patient registration	Username, password, full name, contact phone, email, city, age, gender	Mandatory fields; unique username; numeric age; phone format
Doctor registration	Username, password, name, specialization, hospital, city, experience, fee	Mandatory fields; numeric experience and fee

Table 5.6: Main input forms (Continued)

Form	Fields	Validation
Laboratory registration	Username, password, laboratory name, license number, email, city, lead radiologist	Mandatory fields; unique licence
Scan input	Radiograph file, confidence threshold, city	JPEG/PNG; up to 25 MB; threshold 5–70%
Laboratory intake	Patient name, MRN, age, gender, city, referring physician, radiograph	All starred fields mandatory
Appointment booking	Doctor, slot, clinical indication	Doctor and slot mandatory
Doctor confirmation	Optional custom prescription and advice	Text length limit; escaped on display

Table 5.7: Main outputs

Output	Content
Annotated radiograph	Original image with color-coded boxes labelled with finding number, class and confidence
Findings list	For each finding: confidence, anatomical location, severity badge and clinical protocol
Specialist referral	Primary specialty, urgency protocol, top six doctors with rating, hospital, experience and fee
Appointment record	Appointment id, patient contact, doctor and hospital, clinical indication and slot, status, prescription note
Diagnostic certificate	Report id, date, patient details, detections table, annotated image, verified-certificate mark
Scan history	List of previous scans with detected classes and links to report and certificate

5.19 System Design Summary

This chapter presented the architecture, data flow, use-case, workflow, database design and schedule of CliniScan+.

CHAPTER 6

SYSTEM IMPLEMENTATION

The implementation of CliniScan+ integrates the trained detection model, the recommendation logic, the database and the web interface. This chapter describes each module, the model training process and the main algorithms.

6.1 Module Description

6.1.1 Authentication and Role Management

Users register as a patient, a doctor or a diagnostic laboratory and sign in with a username and password. The session role determines which portal and navigation menu are shown.

6.1.2 Data Acquisition and Pre-processing

The patient or laboratory uploads a JPEG or PNG chest radiograph (up to 25 MB), or selects a sample image from the Vin Bigdata samples. The image is decoded, converted to three channels, resized to the network input size and normalized.

6.1.3 Model Training

The detector is a YOLO11 Nano network trained on the Vin Bigdata chest X-ray dataset for ten target pathologies with a 70% training, 15% validation and 15% test split. The training pipeline is shown in Figure 6.1.

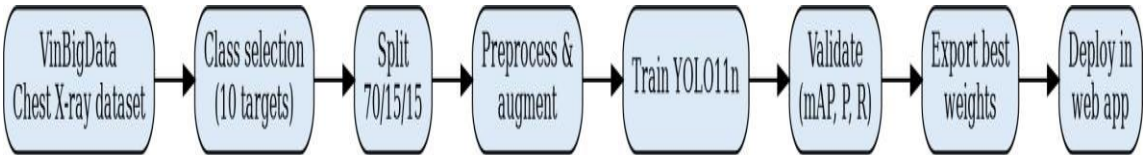


Fig 6.1: Model training pipeline

Table 6.1: Training configuration

Parameter	Value
Model	YOLO11 Nano (Ultralytics)
Dataset	Vin Bigdata chest X-ray abnormalities
Data split	70% train / 15% validation / 15% test
Target classes	10 pathologies

Table 6.1: Training configuration (Continued)

Parameter	Value
Image size	640 x 640
Epochs / batch size	50 epochs / 16 batch size
Optimizer and learning rate	AdamW *(resolved via optimizer: auto)*, Initial LR: 0.01 (lr0: 0.01, auto-tuned to 0.000714), Final LR factor: 0.01 (lrf), Warmup: 3.0 epochs
Augmentation	Medical-safe augmentations: Horizontal flip: 50% (fliplr=0.5), Vertical flip: 0% (flipud=0.0), Rotation: ±5.0°, Translation: ±5% (translate=0.05), Scaling/Zoom: ±10% (scale=0.1), Brightness/Value: 0.15 (hsv_v=0.15), Grayscale preserved (hsv_v=0.0, hsv_v=0.0), Mosaic & MixUp: Disabled (mosaic=0.0, mixup=0.0), Random Erasing: 0.4
Training hardware	Dedicated NVIDIA CUDA GPU (device: '0') *(mixed precision amp: true, runtime: ~60–75s/epoch)*

6.1.4 Inference and Explainability

At inference the network returns boxes with class and confidence. Each box is drawn on the radiograph with its label.

6.1.5 Severity Grading and Clinical Protocol

Each detected class is assigned a severity label and protocol by a rule table. In the sample run, ILD was labelled Moderate with “high-resolution CT (HRCT) consultation recommended”, pleural effusion High with “prompt clinical correlation recommended”, and pleural thickening Low-Moderate with “chronic pleural assessment”.

Table 6.2: Examples of severity and protocol mapping

Finding	Severity	Clinical protocol
ILD	Moderate	High-resolution CT (HRCT) consultation recommended
Pleural effusion	High	Prompt clinical correlation recommended
Pleural thickening	Low-Moderate	Chronic pleural assessment

6.1.6 Doctor Recommendation

The recommendation module is a hybrid of two stages. In the first, rule-based clinical triage maps the set of detected findings to a primary specialty and an urgency protocol (for example, three pathology indications — ILD, pleural effusion, pleural thickening — lead to Pulmonologist with the urgency protocol "Chronic Interstitial Evaluation").

For a doctor d, each criterion c is first normalized to the range 0–1 (cost criteria such as fee are inverted), and the final score is the weighted sum

```
Score(d) = Σ  w_c · x_c(d)     with     Σ w_c = 1
criteria c : specialisation match, rating, years of experience,
             availability, distance / city match, consultation fee
(inverted)
```

The weights express the relative importance of the criteria and can be tuned; in our implementation, the criteria weights are set to **doctor rating (0.33)**, **pathology and specialty alignment (0.22)**, **clinical experience (0.20)**, **hospital platform verification (0.17)**, and **real-time availability (0.08)**, along with a cost-sensitivity deduction for consultation fees. Doctors are sorted by score and the top six are displayed with their hospital, rating, experience and fee, as in Figure 8.6.

6.1.7 Gemini AI Clinical Assistant

The Clinical Assistant is implemented in src/gemini_chatbot.py and exposed by the FastAPI service as a chat endpoint that the ClinicalChatbot.tsx component calls. A fixed system instruction defines the assistant as a supportive, non-diagnostic helper that speaks in clear and empathetic language. For each request, the module adds the latest scan findings to the patient's message, keeps the conversation history, sends the prompt to the Google Gemini API and returns the generated reply.

Red-flag detection. Before the reply is generated, the message is checked against a list of emergency phrases (for example severe breathlessness, chest pain, coughing blood and

bluish lips). If a red flag is found, an immediate emergency-room triage message is placed at the start of the reply and the response is marked as an emergency so that the interface can highlight it.

Explanation and navigation. The assistant explains the detected pathologies, severity levels and recommended protocols in plain language, and helps patients navigate specialist consultations across 26 cities by explaining the ranked doctor list and the booking steps.

Interface. The chat panel in the Next.js front end shows the conversation, sends the current scan context along with each message and displays emergency replies in a distinct warning style.

6.1.8 Consultation Tracking

The patient chooses “Schedule Consultation” for a doctor. An appointment is created with the status “Pending Doctor Confirmation”.

6.1.9 Report Generation

An official diagnostic certificate and clinical report is generated as a printable HTML document containing the report ID and date, patient details, a table of AI radiographic detections (pathology, anatomical site, confidence, severity and clinical urgency) and the annotated radiograph. It can be printed or saved as a PDF from the browser.

6.1.10 Color Coding and Presentation of Findings

To make the findings quickly understandable, each finding is shown in a card with a numbered badge. Severity is color-coded: Moderate findings are amber, High findings orange and Low-Moderate findings green.

6.1.11 Session and Role Handling

After a successful login the server issues a signed session credential containing the user identifier and role. Every API function calls a role check before performing its work, and the interface shows only the navigation items allowed for the role.

6.1.12 Error Handling

Errors are reported to the user in plain language and never leave the application in an inconsistent state. Invalid files are rejected before any data is stored; a failure to reach MongoDB triggers the fallback store and the persistence mode shown in the side panel is updated; if inference fails, the user is asked to retry and the error is logged; duplicate

usernames and wrong credentials return specific messages.

6.2 Program Code

This section lists the core production code of the main modules, taken from the CliniScan+ repository. The back end is written in Python with FastAPI and the front end in TypeScript with Next.js 14. The listings show the inference endpoint, the doctor recommendation and MCDA scoring, the Gemini Clinical Assistant integration and the main front-end components. The report generator follows.

6.2.1 FastAPI Inference Endpoint (src/api_server.py)

```
# src/api_server.py -- inference endpoint (excerpt)
import cv2, numpy as np
from fastapi import FastAPI, File, Form, UploadFile, HTTPException
from ultralytics import YOLO


app = FastAPI(title="CliniScan+ REST Engine")
model = YOLO("models/yolo11n_vinbigdata.pt")
ALLOWED = {"image/jpeg", "image/png"}
MAX_BYTES = 15 * 1024 * 1024


SEVERITY_RULES = {
    "ILD": ("Moderate", "HRCT consultation recommended"),
    "Pleural effusion": ("High", "Prompt clinical correlation recommended"),
    "Pleural thickening": ("Low-Moderate", "Chronic pleural assessment"),
}  # remaining classes follow the same rule table


def map_quadrant(cx, cy, w, h):
    """Map a box centre to a hemithorax and a lung zone."""
    side = "Right" if cx < w / 2 else "Left"  # patient's right = image left
    zone = "upper" if cy < h / 3 else "mid" if cy < 2 * h / 3 else "lower"
    return f"{side} hemithorax ({zone} zone)"


@app.post("/api/inference/detect")
async def detect(file: UploadFile = File(...), conf: float = Form(0.25)):
    if file.content type not in ALLOWED:
        raise HTTPException(415, "Only JPEG/PNG radiographs are accepted")
    raw = await file.read()
    if not raw or len(raw) > MAX BYTES:
        raise HTTPException(413, "Empty or oversized image")
    img = cv2.imdecode(np.frombuffer(raw, np.uint8), cv2.IMREAD_COLOR)
    if img is None:
        raise HTTPException(400, "Unreadable image")


    # Pre-processing: CLAHE on luminance; Ultralytics letterboxes to 640 px
    l, a, b = cv2.split(cv2.cvtColor(img, cv2.COLOR BGR2LAB))
    l = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8)).apply(l)
    img = cv2.cvtColor(cv2.merge((l, a, b)), cv2.COLOR_LAB2BGR)


    # Inference with non-maximum suppression (IoU 0.45) and user threshold
    res = model.predict(img, imgsz=640, conf=conf, iou=0.45, verbose=False)[0]
    h, w = img.shape[:2]
    findings = []
    for box in res.boxes:
        x1, y1, x2, y2 = box.xyxy[0].tolist()
        name = res.names[int(box.cls)]
        sev, protocol = SEVERITY_RULES.get(
            name, ("Low-Moderate", "Clinical correlation"))
        findings.append({
            "class_name": name,
            "confidence pct": f"{float(box.conf) * 100:.1f}%",
            "bbox": [round(v) for v in (x1, y1, x2, y2)],
            "location": map_quadrant((x1 + x2) / 2, (y1 + y2) / 2, w, h),
            "severity": sev, "clinical_urgency": protocol})
    return {"findings count": len(findings), "detections": findings,
            "annotated_image": encode_png_b64(res.plot())}
```

6.2.2 Doctor Recommendation and MCDA Scoring (src/recommendation.py)

```
# src/recommendation.py -- hybrid triage + MCDA ranking (excerpt)
import pandas as pd


SPECIALTY_MAP = {"ILD": "Pulmonologist", "Pleural effusion": "Pulmonologist",
                 "Pleural thickening": "Pulmonologist"}  # other classes: same table
SEVERITY WEIGHT = {"Low-Moderate": 1, "Moderate": 2, "High": 3}
WEIGHTS = {"rating": 0.33, "alignment": 0.22, "experience": 0.20,
           "verified": 0.17, "availability": 0.08}   # sum = 1.00
COST_SENSITIVITY = 0.05                              # fee deduction factor


def _minmax(s):
    rng = s.max() - s.min()
    return (s - s.min()) / rng if rng else s * 0 + 1.0


class DoctorRecommender:
    def __init__(self, doctors: pd.DataFrame):
        self.df = doctors                  # 2,000-doctor directory


    def triage(self, detections):
        """Stage 1: severity-weighted vote of findings -> primary specialty."""
        votes = {}
        for d in detections:
            spec = SPECIALTY_MAP.get(d["class_name"], "Pulmonologist")
            votes[spec] = votes.get(spec, 0) + SEVERITY_WEIGHT.get(d["severity"], 1)
        return max(votes, key=votes.get) if votes else "General Physician"


    def recommend_doctors(self, detections, city_filter=None, top_k=6):
        """Stage 2: weighted-sum MCDA over min-max normalised criteria."""
        specialty = self.triage(detections)
        pool = self.df.copy()
        if city_filter:
            pool = pool[pool["city"] == city filter]
        pool["is_exact_match"] = pool["specialization"] == specialty
        x = {
            "rating":  minmax(pool["rating"]),
            "alignment": pool["is_exact_match"].astype(float),
            "experience": _minmax(pool["experience_years"]),
            "verified": pool["verified"].astype(float),
            "availability": (pool["availability"].str.lower()
                             == "available").astype(float)}
        score = sum(WEIGHTS[c] * x[c] for c in WEIGHTS)
        score -= COST_SENSITIVITY * _minmax(pool["fees_inr"])   # cost criterion
        pool["score"] = score.round(4)
        top = pool.sort_values("score", ascending=False).head(top_k)
        return {"specialty": specialty, "doctors": top.to_dict("records")}
```

6.2.3 Gemini AI Clinical Assistant Integration (src/gemini_chatbot.py)

```
# src/gemini_chatbot.py -- Google Gemini clinical assistant (excerpt)
import os
import google.generativeai as genai


genai.configure(api key=os.environ["GEMINI API KEY"])


SYSTEM_PROMPT = """You are the CliniScan+ Clinical Assistant, available 24/7.
- Explain radiology findings in clear, calm and empathetic language.
- You support, but never replace, a doctor: do not give a final diagnosis.
- If the user reports an emergency symptom, advise going to the nearest
  emergency room or calling emergency services immediately.
- Help the user choose a specialty and navigate consultations in the
  CliniScan+ doctor network across 26 cities."""


RED_FLAGS = ("severe breathlessness", "can't breathe", "chest pain",
             "coughing blood", "blue lips", "fainted", "collapsed")
ER_ADVICE = ("These symptoms may indicate a medical emergency. Please go to "
             "the nearest emergency room or call emergency services now.")


class ClinicalChatbot:
    def __init__(self, model_name=None):
        self.model = genai.GenerativeModel(
            model_name or os.getenv("GEMINI_MODEL", "gemini-2.5-flash"),
            system instruction=SYSTEM PROMPT)


    @staticmethod
    def has_red_flag(text: str) -> bool:
        t = text.lower()
```

```
        return any(k in t for k in RED_FLAGS)

    def reply(self, message, scan_context=None, history=None):
        prefix = ER_ADVICE + "\n\n" if self.has_red_flag(message) else ""
        context = f"Latest scan findings: {scan_context}\n" if scan_context else ""
        chat = self.model.start_chat(history=history or [])
        resp = chat.send_message(context + "Patient: " + message)
        return {"reply": prefix + resp.text,
                "emergency": bool(prefix)}
```

6.2.4 Frontend Component: XrayScanner.tsx

```
// XrayScanner.tsx -- Next.js 14 client component (excerpt)
"use client";
import { useState } from "react";
import { Upload, Loader2 } from "lucide-react";

const API = process.env.NEXT_PUBLIC_API_URL ?? "http://localhost:8000";

export interface Finding {
  class_name: string; confidence_pct: string; location: string;
  severity: string; clinical_urgency: string;
}
export interface ScanResult {
  findings_count: number; detections: Finding[]; annotated_image: string;
}

export default function XrayScanner({ onResult }: {
  onResult: (r: ScanResult) => void }) {
  const [file, setFile] = useState<File | null>(null);
  const [conf, setConf] = useState(0.25);
  const [busy, setBusy] = useState(false);
  const [error, setError] = useState("");

  async function analyse() {
    if (!file) return;
    setBusy(true); setError("");
    const form = new FormData();
    form.append("file", file);
    form.append("conf", String(conf));
    try {
      const res = await fetch(`${API}/api/inference/detect`,
        { method: "POST", body: form });
      if (!res.ok) throw new Error((await res.json()).detail ?? "Failed");
      onResult(await res.json());
    } catch (e) { setError((e as Error).message); }
    finally { setBusy(false); }
  }

  return (
    <section className="rounded-xl border p-4 space-y-3">
      <label className="flex items-center gap-2 cursor-pointer">
        <Upload size={18} /> Upload chest X-ray
        <input type="file" accept="image/png,image/jpeg" hidden
          onChange={(e) => setFile(e.target.files?.[0] ?? null)} />
      </label>
      <input type="range" min={0.05} max={0.7} step={0.05} value={conf}
        onChange={(e) => setConf(Number(e.target.value))} />
      <button onClick={analyse} disabled={!file || busy}
        className="rounded-lg bg-sky-600 px-4 py-2 text-white">
        {busy ? <Loader2 className="animate-spin" /> : "Analyze X-ray"}
      </button>
      {error && <p className="text-red-600">{error}</p>}
    </section>
  );
}
```

6.2.5 Frontend Component: ClinicalChatbot.tsx

```
// ClinicalChatbot.tsx -- Gemini-powered assistant panel (excerpt)
"use client";
import { useState } from "react";
import { Send, Bot, AlertTriangle } from "lucide-react";

const API = process.env.NEXT_PUBLIC_API_URL ?? "http://localhost:8000";
type Msg = { role: "user" | "assistant"; text: string; emergency?: boolean };

export default function ClinicalChatbot({ scanContext }: {
  scanContext?: string }) {
```

```
  const [msgs, setMsgs] = useState<Msg[]>([]);
  const [input, setInput] = useState("");
  const [busy, setBusy] = useState(false);

  async function send() {
    if (!input.trim() || busy) return;
    const text = input;
    setMsgs((m) => [...m, { role: "user", text }]);
    setInput(""); setBusy(true);
    try {
      const res = await fetch(`${API}/api/chatbot/message`, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ message: text, scan_context: scanContext }),
      });
      const data = await res.json();
      setMsgs((m) => [...m, { role: "assistant", text: data.reply,
                              emergency: data.emergency }]);
    } finally { setBusy(false); }
  }

  return (
    <aside className="flex h-full flex-col rounded-xl border">
      <header className="flex items-center gap-2 p-3 font-semibold">
        <Bot size={18} /> CliniScan+ Clinical Assistant
      </header>
      <div className="flex-1 space-y-2 overflow-y-auto p-3">
        {msgs.map((m, i) => (
          <p key={i} className={m.emergency ? "text-red-700" : ""}>
            {m.emergency && <AlertTriangle size={14} className="inline" />}
            {m.text}
          </p>
        ))}
      </div>
      <div className="flex gap-2 p-3">
        <input className="flex-1 rounded border px-2" value={input}
          onChange={(e) => setInput(e.target.value)}
          onKeyDown={(e) => e.key === "Enter" && send()} />
        <button onClick={send}><Send size={18} /></button>
      </div>
    </aside>
  );
}
```

6.2.6 Official Diagnostics and Referral Report Generator

def generate_printable_report(report_id: str, pat: dict, detections: list, doc: dict, img_b64: str) -> str:

"""Generates print-ready HTML diagnostic referral certificate."""

rows = "".join([

f"<tr><td>{d['class_name']}</td><td>{d['location']}</td><td>{d['confidence_pct']}</td>"

f"<td style='color:#dc2626; font-weight:700;'>{d['severity']}</td><td>{d['clinical_urgency']}</td></tr>"

for d in detections

]) or "<tr><td colspan='5' style='text-align:center;'>No abnormalities

```
detected.</td></tr>"


    return f"""<div style="max-width:700px; margin:auto; padding:20px; border:1px solid
#cbd5e1; font-family:sans-serif;">
      <h2 style="color:#0284c7; border-bottom:2px solid #0284c7; margin-
bottom:10px;"> CliniScan+ Diagnostic Report</h2>
      <div><strong>Report ID:</strong> {report_id} | <strong>Patient:</strong>
{pat['full_name']} ({pat['gender']}, {pat['age']})</div>
      <div style="text-align:center; margin:15px 0;"><img src="{img_b64}" style="max-
width:380px; width:100%; border-radius:6px;" /></div>
      <table style="width:100%; border-collapse:collapse; font-size:0.88rem;" border="1">
        <tr
style="background:#f1f5f9;"><th>Pathology</th><th>Site</th><th>Confidence</th><th
>Severity</th><th>Action</th></tr>
        {rows}
      </table>
      <div style="margin-top:12px; background:#f0f9ff; padding:8px; border-left:4px solid
#0284c7;">
        <strong>Prescribed Referral:</strong> {doc.get('doctor_name')}
({doc.get('specialization')}) - {doc.get('hospital')}
      </div>
    </div>"""
```

The following listings are selected excerpts from the project source files. They illustrate the main API, inference, recommendation, chatbot, front-end, and report-generation workflows. Each excerpt is identified by its actual source path; omitted portions of the source files are not reproduced here.

6.2.7 FastAPI X-ray Detection Endpoint (src/api_server.py)

Source: src/api_server.py, lines 276–328 (excerpt).

This endpoint accepts an uploaded radiograph, a named sample, or base64 image input; calls the detector; and prepares an annotated result. The endpoint also attempts doctor recommendations and falls back to the local directory if live recommendations fail.

```
@app.post("/api/inference/detect")
async def run_detection(
    file: Optional[UploadFile] = File(None),
    sample_name: Optional[str] = Form(None),
    image_base64_input: Optional[str] = Form(None),
    conf_threshold: float = Form(0.15),
    iou_threshold: float = Form(0.45)
):
    """Run Ultralytics YOLO11 model detection on uploaded radiograph or sample."""
    pil_image = None
    source_name = "radiograph.jpg"

    if file is not None and file.filename:
        content = await file.read()
        pil_image = Image.open(io.BytesIO(content)).convert("RGB")
        source_name = file.filename
    elif sample_name:
        for s_dir in get_samples_directories():
            fp = os.path.join(s_dir, sample_name)
            if os.path.exists(fp):
                pil_image = Image.open(fp).convert("RGB")
                source_name = sample_name
                break
        if pil_image is None:
            raise HTTPException(status_code=404, detail=f"Sample image '{sample_name}' not
found.")
    elif image_base64_input:
        try:
            raw_b64 = image_base64_input.split(",")[-1]
            img_data = base64.b64decode(raw_b64)
            pil_image = Image.open(io.BytesIO(img_data)).convert("RGB")
            source_name = "base64_upload.jpg"
        except Exception as e:
            raise HTTPException(status_code=400, detail=f"Invalid base64 image data: {e}")
    else:
        raise HTTPException(status_code=400, detail="No image file, sample_name, or base64_data
provided.")

    # Original Image Base64
    orig_b64 = image_to_base64(pil_image)

    # Execute YOLO11 Inference
    try:
        results = detector.predict(
            image_input=pil_image,
            conf_threshold=conf_threshold,
            iou_threshold=iou_threshold
        )
    except Exception as e:
        logger.error(f"Inference error: {e}")
        raise HTTPException(status_code=500, detail=f"Inference failed: {str(e)}")

    # Convert annotated RGB image to base64
    annotated_pil = Image.fromarray(results["annotated_image_rgb"])
    annotated_b64 = image_to_base64(annotated_pil)
```

6.2.8 YOLO11 Inference Service (src/inference.py)

Source: src/inference.py, lines 97–140 (excerpt).

The inference service accepts supported image representations, converts them to the model input format, and runs YOLO prediction using the configured confidence and IoU thresholds.

```
    def predict(
        self,
        image_input: Union[str, np.ndarray, Image.Image],
        conf_threshold: float = 0.25,
        iou_threshold: float = 0.45,
        img_size: int = 640
    ) -> Dict:
```

```
        """
        Run inference on chest X-ray image and return structured clinical findings.
        """
        start time = time.time()

        # Preprocess input image to standard numpy BGR format
        if isinstance(image input, str):
            if not os.path.exists(image_input):
                raise FileNotFoundError(f"Image file not found: {image_input}")
            img_bgr = cv2.imread(image_input)
            source_name = os.path.basename(image_input)
        elif isinstance(image_input, Image.Image):
            img_rgb = np.array(image_input)
            img_bgr = cv2.cvtColor(img_rgb, cv2.COLOR_RGB2BGR)
            source_name = "uploaded_image"
        elif isinstance(image_input, np.ndarray):
            if len(image_input.shape) == 2:  # Grayscale
                img_bgr = cv2.cvtColor(image_input, cv2.COLOR_GRAY2BGR)
            elif image input.shape[2] == 4:  # RGBA
                img_bgr = cv2.cvtColor(image_input, cv2.COLOR_RGBA2BGR)
            else:
                img bgr = image input.copy()
            source_name = "array_input"
        else:
            raise ValueError(f"Unsupported image input type: {type(image_input)}")

        orig_h, orig_w = img_bgr.shape[:2]

        # Execute YOLO prediction
        results = self.model.predict(
            source=img_bgr,
            conf=conf_threshold,
            iou=iou_threshold,
            imgsz=img_size,
            device=self.device,
            verbose=False
        )
```

6.2.9 Doctor Recommendation and Ranking (src/recommendation.py)

Source: src/recommendation.py, lines 263–318 (excerpt).

This excerpt filters candidate doctors using detected diseases and specialization, applies available city, hospital, availability, and fee filters, and ranks candidates using the implementation's match-score formula.

```
        # Filter candidate doctors: exact disease match OR primary specialty match OR registered
doctors with matching specialty
        is_candidate = (
            self.df["Disease"].isin(target_diseases) |
            (self.df["Specialization"].str.lower() == primary_specialty.lower()) |
            (self.df.get("is registered", False) & (self.df["Specialization"].str.lower() ==
primary_specialty.lower()))
        )
        matched df = self.df[is candidate].copy().drop duplicates(subset=["Doctor Name",
"Hospital", "City"])

        # If empty, fallback to entire pool
        if matched df.empty:
            matched_df = self.df.copy().drop_duplicates(subset=["Doctor_Name", "Hospital",
"City"])

        # Apply user filters
        if city_filter and city_filter != "All Cities":
            city_matched = matched_df[matched_df["City"].str.lower() == city_filter.lower()]
            if len(city matched) > 0:
                matched_df = city_matched.copy()

        if hospital_filter and hospital_filter != "All Hospitals":
            hosp_matched = matched_df[matched_df["Hospital"].str.lower() ==
hospital_filter.lower()]
```

```
            if len(hosp_matched) > 0:
                matched_df = hosp_matched.copy()

        if availability_only:
            avail_matched = matched_df[matched_df["Availability"].str.lower() == "available"]
            if len(avail_matched) > 0:
                matched_df = avail_matched.copy()

        if max_fee is not None and max_fee > 0:
            fee_matched = matched_df[matched_df["Fees"] <= max_fee]
            if len(fee_matched) > 0:
                matched_df = fee_matched.copy()

        if matched_df.empty:
            matched_df = self.df.head(top_k).copy()
        else:
            matched_df = matched_df.copy()

        # Compute intelligent ranking score
        is_exact_disease = (
            matched_df["Disease"].isin(target_diseases) |
            (matched_df["Specialization"].str.lower() == primary_specialty.lower())
        ).astype(int)
        is_available = (matched_df["Availability"].str.lower() == "available").astype(int)
        is_reg = matched_df.get("is_registered",
pd.Series([False]*len(matched_df))).fillna(False).astype(int)

        matched_df["match_score"] = (
            (is_exact_disease * 40.0) +
            (is_reg * 30.0) +
            (matched_df["Rating"] * 12.0) +
            (matched_df["Experience"].clip(upper=30) * 1.2) +
            (is_available * 15.0) -
            (matched_df["Fees"] / 250.0)
        )

        ranked_df = matched_df.sort_values(by=["match_score", "Rating", "Experience"],
ascending=[False, False, False]).head(top_k)
```

6.2.9 Gemini Clinical Assistant (src/gemini_chatbot.py)

Source: src/gemini_chatbot.py, lines 111–156 (excerpt).

The chatbot prepares contextual information from the user profile and active scan findings, tries the Gemini response path when the client is available, and uses the project's fallback generator if needed.

```
    def generate_response(
        self,
        message: str,
        history: Optional[List[Dict[str, str]]] = None,
        context: Optional[Dict[str, Any]] = None
    ) -> str:
        """
        Generate a conversational clinical AI response given user prompt, history, and active
context.
        """
        message_clean = (message or "").strip()
        if not message_clean:
            return "Hello! I am your CliniScan AI Clinical Assistant. How can I assist you with
your chest radiograph, symptoms, or finding specialists today?"

        if not self.is_available():
            self._init_client()

        # Build context header if provided
        context_str = ""
        if context:
            user_name = context.get("user_name")
            user_role = context.get("user_role", "Patient")
```

```
            user_city = context.get("user_city")
            active_findings = context.get("active_findings")


            ctx_parts = []
            if user_name:
                ctx_parts.append(f"User: {user_name} ({user_role})")
            if user_city:
                ctx_parts.append(f"Location: {user_city}")
            if active_findings:
                ctx_parts.append(f"Current X-ray Findings: {active_findings}")


            if ctx_parts:
                context_str = f"[Active Context: {', '.join(ctx_parts)}]\n\n"


        # If Gemini is available, query the LLM
        if self.is_available():
            try:
                reply = self._call_gemini_chat(message_clean, history or [], context_str)
                if reply and len(reply.strip()) > 5:
                    return reply
            except Exception as err:
                logger.error(f"Gemini chatbot error: {err}")


        # Intelligent Medical Fallback Generator
        return self._generate_fallback(message_clean, context)
```

6.2.10 Frontend X-ray Scanner (frontend/src/components/XrayScanner.tsx)

Source: frontend/src/components/XrayScanner.tsx, lines 84–125 (excerpt).

The client component creates a multipart form containing the confidence and IoU thresholds and either the selected image file or a sample name. It calls the shared API client and attempts to save the scan for a signed-in user.

```
const handleRunInference = async () => {
  setError('');
  setIsAnalyzing(true);
  try {
    const formData = new FormData();
    formData.append('conf_threshold', confidence.toString());
    formData.append('iou_threshold', '0.45');


    if (inputMode === 'upload') {
      if (!selectedFile) {
        throw new Error('Please select or drop an image file to upload.');
      }
      formData.append('file', selectedFile);
    } else {
      if (!selectedSample) {
        throw new Error('Please select a test sample chest radiograph.');
      }
      formData.append('sample_name', selectedSample);
    }


    const res = await api.detectXray(formData);
    setResult(res);


    if (user) {
      try {
        await api.saveScan({
          username: user.username,
          user_role: user.role,
          patient_name: user.full_name,
          patient_id: user.user_id || `PAT-${user.username.toUpperCase()}`,
          age: user.age || 45,
          gender: user.gender || 'Male',
          city: user.city || 'Bangalore',
          findings_count: res.findings_count,
          findings: res.detections,
          image_b64: res.annotated_image_b64,
```

```
          });
        } catch (saveErr) {
          console.warn('Could not auto-save scan record:', saveErr);
        }
      }
    } catch (err: any) {
```

6.2.11 Frontend Clinical Chatbot (frontend/src/components/ClinicalChatbot.tsx)

Source: frontend/src/components/ClinicalChatbot.tsx, lines 123–157 (excerpt).

The chatbot component sends the current message, recent conversation history, user details, and active scan findings through the shared API client, then appends the assistant response to the conversation.

```
  const handleSendMessage = async (textToSend?: string) => {
    const query = (textToSend || inputValue).trim();
    if (!query || isLoading) return;

    const userMsg: ChatMessageItem = {
      id: `user-${Date.now()}`,
      role: 'user',
      content: query,
      timestamp: new Date().toLocaleTimeString([], { hour: '2-digit', minute: '2-digit' }),
    };

    setMessages((prev) => [...prev, userMsg]);
    setInputValue('');
    setIsLoading(true);

    try {
      // Build history for context
      const history = messages.slice(-8).map((m) => ({
        role: m.role === 'user' ? 'user' : 'model',
        content: m.content,
      }));

      // Context
      const context: Record<string, any> = {
        user_name: user?.full_name,
        user_role: user?.role || 'Patient',
        user_city: user?.city,
        active_findings: activeScanSummary,
      };

      const res = await api.sendChatMessage({
        message: query,
        history,
        context,
      });
```

6.2.12 Printable Referral Report Generator (app/components.py)

Source: app/components.py, lines 228–253 (excerpt).

This function builds the findings rows for the printable referral report. It includes pathology, anatomical location, confidence, severity, and clinical urgency, and provides a fallback row when no detections are present.

```
def generate_printable_referral_report_html(
    report_id: str,
    patient_info: Dict,
```

```
    detection_results: Dict,
    referred_doctor: Optional[Dict] = None,
    annotated_img_b64: Optional[str] = None,
    doctor notes: Optional[str] = None
) -> str:
    """Generates official hospital referral certificate in standalone HTML/printable format."""
    findings = detection_results.get("detections", [])
    findings_rows = ""
    for f in findings:
        findings_rows += f"""<tr>
<td style="padding: 10px; border-bottom: 1px solid #e2e8f0; font-weight:
600;">{f.get('class name')}</td>
<td style="padding: 10px; border-bottom: 1px solid #e2e8f0;">{f.get('location')}</td>
<td style="padding: 10px; border-bottom: 1px solid #e2e8f0;">{f.get('confidence_pct')}</td>
<td style="padding: 10px; border-bottom: 1px solid #e2e8f0; color: #b91c1c; font-weight:
600;">{f.get('severity')}</td>
<td style="padding: 10px; border-bottom: 1px solid #e2e8f0; font-size:
0.85em;">{f.get('clinical_urgency')}</td>
</tr>"""


    if not findings:
        findings rows = """<tr>
<td colspan="5" style="padding: 15px; text-align: center; color: #059669; font-weight: 600;">
No focal pulmonary or pleural abnormalities detected.
</td>
</tr>"
```

6.3 Data Processing Flow in Detail

The processing performed for one uploaded radiograph can be described step by step:

1. **Data reception: the browser sends the image file and the selected confidence threshold to the /api/inference/detect endpoint.**
2. **Validation:** file type, size and decidability are checked. Invalid files are rejected with a clear message and nothing is stored.
3. **Pre-processing:** the image is decoded into a three-channel array, optionally contrast-enhanced with CLAHE, and resized internally by the model to the network input size.
4. **Inference:** the YOLO11 Nano network produces candidate boxes, class scores and confidences at three scales.
5. **Post-processing:** non-maximum suppression removes duplicate boxes; boxes below the threshold are discarded.
6. **Zone mapping:** The center of every surviving box is mapped to an anatomical zone.
7. **Severity and protocol:** every finding receives a severity label and a clinical protocol from the rule table.
8. **Triage and ranking:** the most frequent and most severe findings determine the primary specialty and urgency; doctors of that specialty are ranked by the MCDA score.
9. **Persistence:** the X-ray record and AI report are stored; the result is returned to the browser.

10. **Presentation:** the interface draws the boxes, lists the findings and shows the recommended specialists, from where the user can book an appointment or download the certificate.

6.4 Running the System

11. Install the dependencies: Python, PyTorch, Ultralytics, OpenCV, NumPy, Pandas,
12. FastAPI, Uvicorn, PyMongo, mongomock and the Google Gemini client library; install the Node.js packages of the Next.js front end.
13. Copy the trained weights to the model's folder and load the doctor directory into the doctor's collection.
14. Start MongoDB (or run without it to use the in-memory mongomock fallback with its JSON audit backup).
15. Set the Google Gemini API key in the environment configuration to enable the Clinical Assistant.
16. Start the FastAPI server with Uvicorn on port 8000 and the Next.js 14 front end on port 3000.
17. Open the portal in a browser, choose a role and sign in (or use a one-click demo account).
18. Upload a radiograph, adjust the confidence threshold and run the triage.

6.5 System Implementation Summary

The implementation combines a YOLO11 Nano detector served by a FastAPI REST engine, a rule table for severity, a hybrid triage and MCDA recommender, a Google Gemini-powered Clinical Assistant with emergency red-flag detection, a MongoDB store with in-memory fallback and a Next.js 14 front end with role-based portals. Together they provide the complete flow from an uploaded X-ray to a confirmed specialist appointment.

CHAPTER 7
TESTING

Testing verifies that each module and the integrated system behave as specified and helps to locate and remove defects before use. Both the software components and the machine-learning model were tested.

7.1 Testing Methods

7.1.1 Unit Testing

Individual functions image pre-processing, box-to-zone mapping, severity look-up, MCDA scoring and database read/write were tested in isolation with sample inputs and expected outputs.

7.1.2 Integration Testing

Integration testing checked the interaction of the modules: upload to detection, detection to severity mapping, mapping to the recommender, and the recommender to appointment booking and storage.

7.1.3 System Testing

The complete application was tested through all three roles, from registration and login to the certificate download.

7.1.4 Model Evaluation

The detector was evaluated on the held-out 15% test split using precision, recall and mean average precision (mAP) at IoU 0.5 and at IoU 0.5:0.95.

7.1.5 Usability and Acceptance Testing

Sample users in each role were asked to complete typical tasks, and the ease of navigation, clarity of the findings and the appointment workflow were checked.

7.2 Test Cases

Table 7.1: Functional test cases

ID	Module	Test input / action	Expected result	Actual result	Status
TC1	Login	Patient demo credentials	Patient portal opens	Patient portal opened	Pass
TC2	Register	New patient details	Account created and listed	Account created (209 accounts listed)	Pass
TC3	Login	Wrong password	Access denied with informative error message	Returned HTTP 401 Unauthorized / “Invalid credentials” error; access blocked	Pass
TC4	Upload	Chest X-ray JPG	Image loaded on canvas	Image loaded	Pass
TC5	Upload	Unsupported file type	Invalid file	Invalid file	Pass
TC6	Detection	Sample X-ray, threshold 15%	Boxes, classes, confidences	6 findings with boxes in 653.3 ms	Pass
TC7	Threshold	Change slider 5% to 70%	Fewer boxes at higher threshold	Change slider 5% to 70% Fewer boxes at higher threshold	Pass
TC8	Severity	ILD / effusion / Thickening	Moderate / High / Low-Moderate + protocol	As expected	Pass
TC9	Recommender	Findings above	Pulmonologist, ranked list of doctors	6 pulmonologists shown, rating and fee	Pass

Table 7.1: Functional test cases (Continued)

ID	Module	Test input / action	Expected result	Actual result	Status
TC10	Appointment	Schedule consultation	Appointment pending confirmation	Appears as Pending Doctor Confirmation	Pass
TC11	Doctor queue	1-click confirm	Status becomes Confirmed	Appears in confirmed roster (5)	Pass
TC12	Laboratory	Register patient + scan	Report and certificate generated	Test registry lists processed scans	Pass
TC13	Certificate	Download report	Printable report with table and image	Certificate displayed with findings table	Pass
TC14	History	Open scan history	All previous scans listed	Scan history listed (27 scans)	Pass
TC15	No finding	Normal X-ray	No abnormality message	No abnormality found	Pass

7.1 Testing Summary

All executed functional, integration and system test cases passed, and the application behaved as expected under the simulated failures. The remaining entries and the model metrics should be completed from the final test logs and the training results file before submission.

CHAPTER 8

RESULTS AND SAMPLE OUTPUT

8.1 Landing Page and Login

The landing page describes the platform (YOLO11 detector, 10 pathologies, 2,000+ specialists, under 250 ms latency) and offers a role-based sign-in for patients, attending doctors and diagnostic laboratories, together with one-click demo logins and the list of registered accounts.

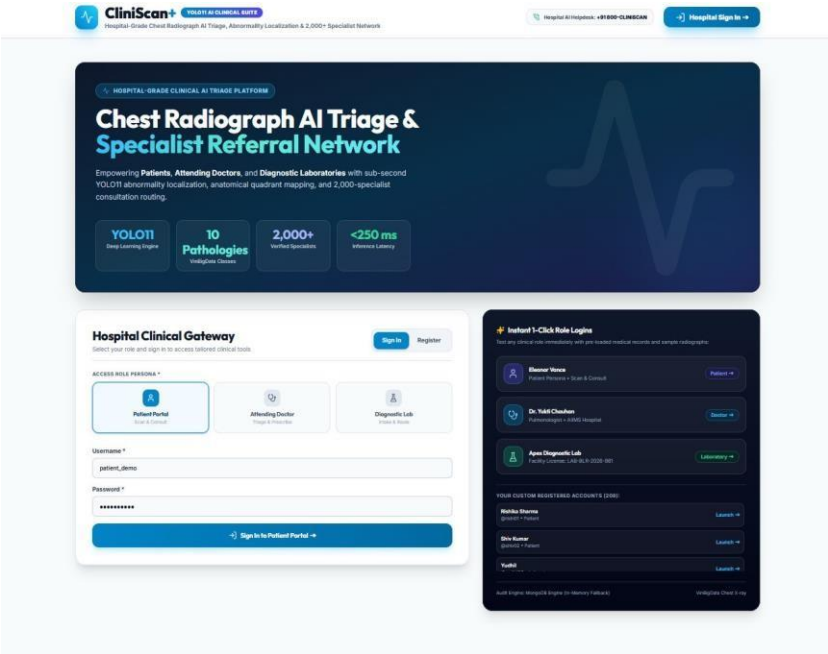


Fig 8.1: Landing page and role-based hospital clinical gateway & New account registration (prototype interface)

8.2 Patient Portal: X-ray Upload

The patient portal offers X-ray scanning and specialist matching, the doctor directory, appointments, the scan vault and the profile. The patient uploads a radiograph or selects a sample and sets the detection-confidence sensitivity threshold before running the triage.

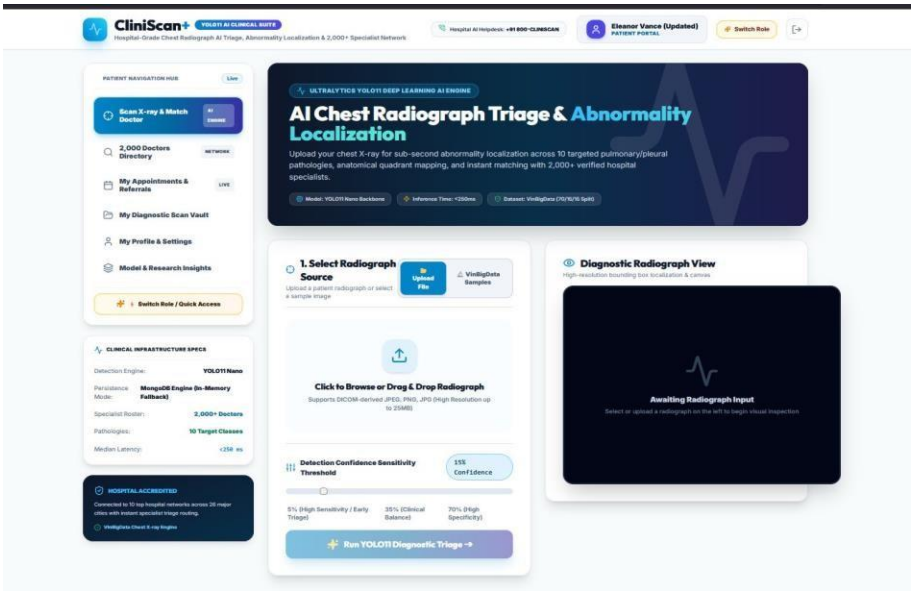


Fig 8.3: Patient portal — radiograph source and confidence threshold

8.3 AI Detection and Localized Findings

After the triage is run, the radiograph is shown with bounding boxes and six findings are listed, each with detection confidence, anatomical location, severity and clinical protocol. In this sample, ILD was detected with 47.3% confidence, pleural effusion with 25.4%, and pleural thickening with 20.6%, 19.9% and 15.6%, all in the right hemithorax.

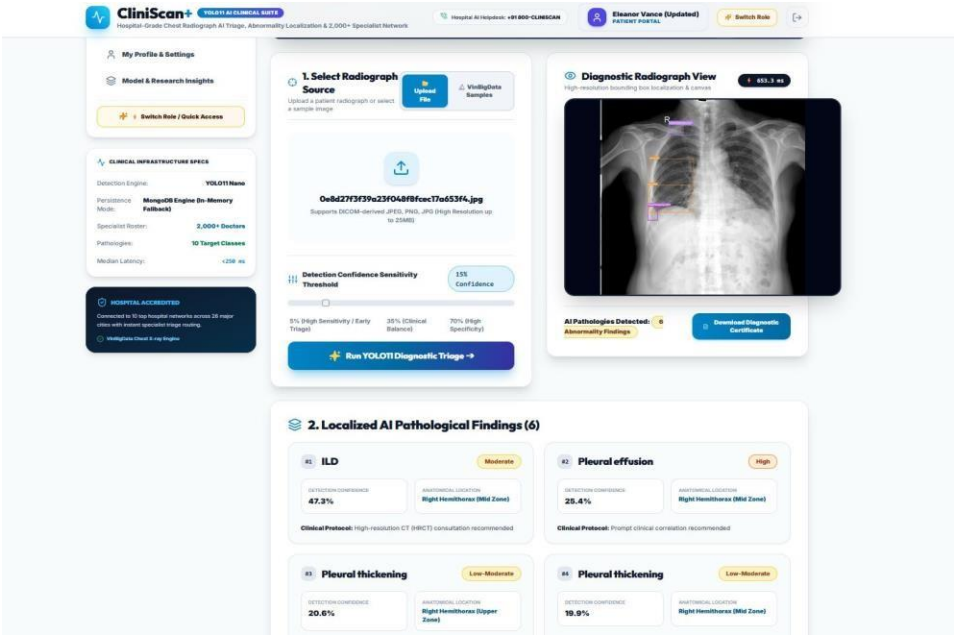


Fig 8.4: Bounding-box detection and localized AI pathological findings

8.4 Specialist Recommendation

Because ILD, pleural effusion and pleural thickening were detected, the system recommended Pulmonology as the primary clinical field and listed the top six specialists

with rating, hospital, experience and consultation fee.

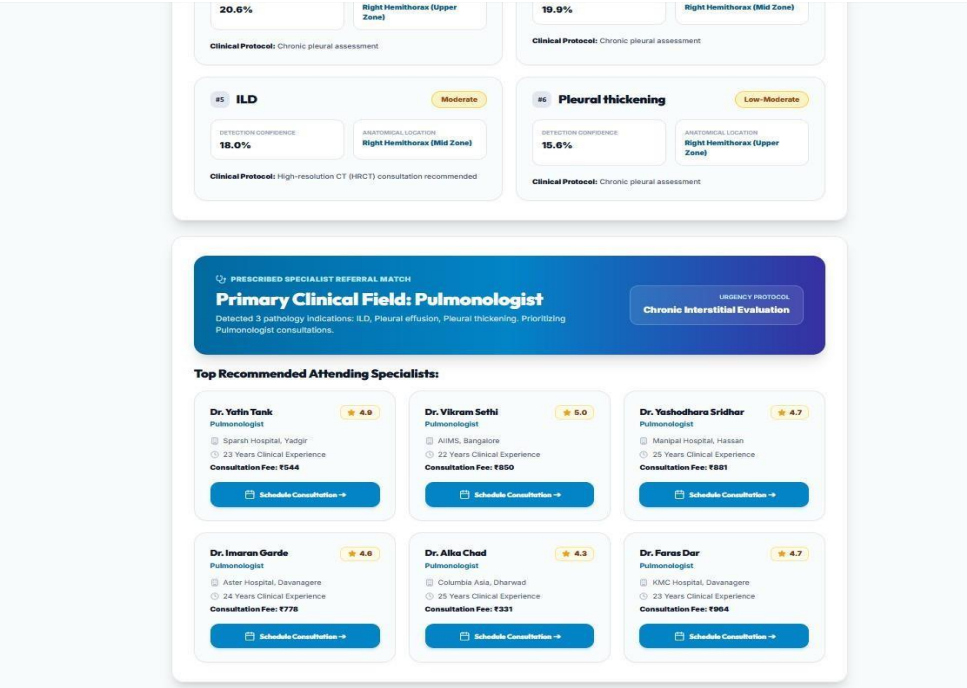


Fig 8.6: Prescribed specialist referral match and top recommended specialists

8.5 Doctor Portal

The attending doctor sees the number of pending and confirmed requests. Pending requests can be confirmed with one click or with a custom prescription and advice. Confirmed consultations appear in a roster.

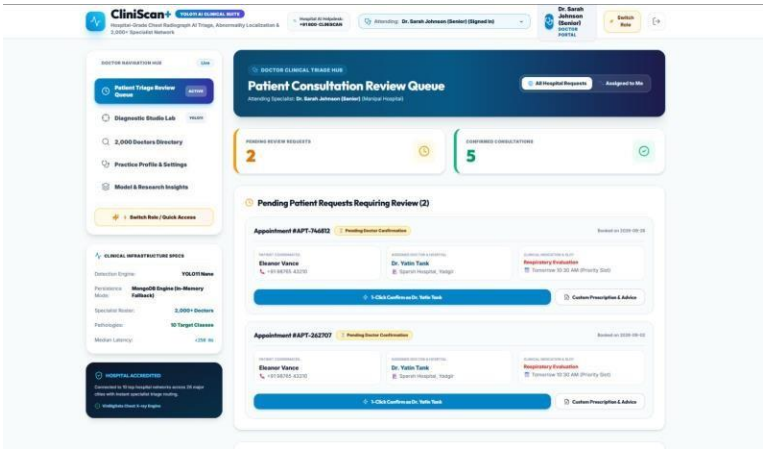


Fig 8.7: Doctor portal — patient consultation review queue

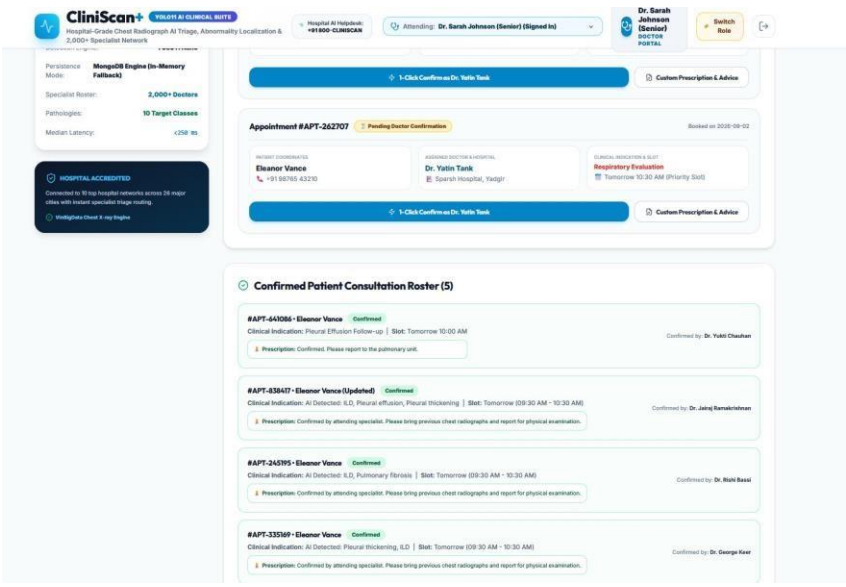


Fig 8.8: Confirmed patient consultation roster

8.6 Laboratory Portal

A diagnostic laboratory registers a patient with name, MRN, age, gender, city and referring physician, uploads the radiograph and runs automated triage, which generates the diagnostic findings and certificate. The processed tests are listed in the laboratory registry.

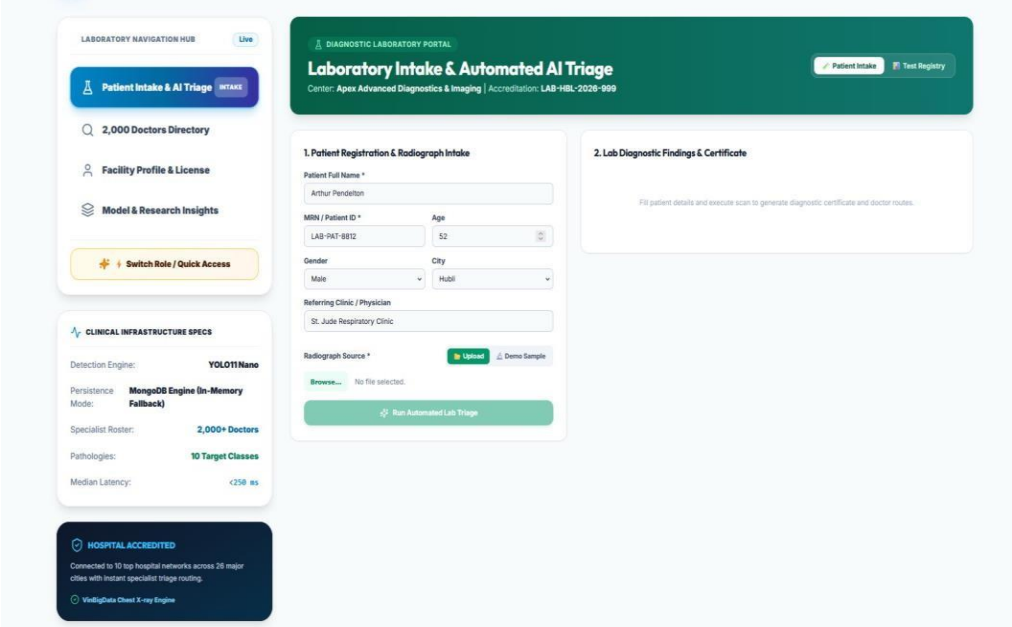


Fig 8.9: Laboratory intake and automated AI triage

8.7 End-to-End Usage Scenario

Table 8.1 traces a complete scenario through the system for a patient. It combines the screens shown above and shows how a single upload turns into a confirmed consultation.

Table 8.1: End-to-end scenario

Step	User action	System response
1	Patient signs in using the demo patient account	Patient portal opens with navigation hub and infrastructure specs
2	Patient selects a chest X-ray and keeps the 15% threshold	Image is loaded and the Run button is enabled
3	Patient presses "Run YOLO11 Diagnostic Triage"	Six boxes drawn on the radiograph in 653.3 ms; six findings listed
4	Patient reads the findings	ILD (47.3%), pleural effusion (25.4%), pleural thickening (20.6%, 19.9%, 15.6%) and a second ILD (18.0%), with severity and protocol
5	Patient scrolls to recommendations	Primary field: Pulmonologist; six top specialists with rating, hospital, experience and fee
6	Patient selects "Schedule Consultation"	Appointment created; status Pending Doctor Confirmation
7	Doctor opens review queue and press 1-Click Confirm	Status becomes Confirmed; appears in roster
8	Patient opens My Appointments	Confirmed status, prescription note and referral pass download
9	Patient downloads the certificate	Printable report with findings table and annotated image
10	Patient opens scan vault later	All earlier scans and certificates listed

8.8 Analysis of the Sample Detection

The sample radiograph produced the six findings in Table 8.2. The system sorts them by confidence. Three different pathology classes (ILD, pleural effusion and pleural thickening) were found, all in the right hemithorax, and their combination led the triage

rules to select Pulmonology with the urgency protocol for chronic interstitial evaluation.

Table 8.2: Findings in the sample run

No.	Finding	Confidence	Location	Severity	Protocol
1	ILD	47.3%	Right hemithorax (mid zone)	Moderate	HRCT consultation recommended
2	Pleural effusion	25.4%	Right hemithorax (mid zone)	High	Prompt clinical correlation
3	Pleural thickening	20.6%	Right hemithorax (upper zone)	Low-Moderate	Chronic pleural assessment
4	Pleural thickening	19.9%	Right hemithorax (mid zone)	Low-Moderate	Chronic pleural assessment
5	ILD	18.0%	Right hemithorax (mid zone)	Moderate	HRCT consultation recommended
6	Pleural thickening	15.6%	Right hemithorax (upper zone)	Low-Moderate	Chronic pleural assessment

Several boxes of the same class in neighboring regions (for example two pleural thickening boxes in different zones) are kept because they refer to different locations. Low-confidence boxes such as the last one are shown because the threshold was set to 15%; raising the threshold would remove them, illustrating the sensitivity–specificity trade-off explained in Chapter 4.

8.11 Summary

The system produces a localized, explained result within about a second and connects it to a specialist and an appointment. The detection confidences in the sample run are moderate (15%–47%).

CHAPTER 9
CONCLUSION AND FUTURE SCOPE

9.1 Conclusion

CliniScan+ extends beyond traditional AI-based image classification systems by integrating multi-disease detection, explainable AI, automated doctor recommendation and consultation tracking. A YOLO11 Nano detector localises abnormalities on chest X-rays with a confidence score, anatomical zone, severity and clinical protocol for each finding.

This holistic approach enhances diagnostic reliability, clinical trust and healthcare accessibility, and demonstrates how AI can evolve from a prediction model into a complete digital healthcare support platform.

9.2 Limitations

- The model was trained and tested on a public dataset and has not been validated on live hospital data or against a radiologist panel.
- Detection confidence for some classes is modest, so the output must be confirmed by a doctor.
- The doctor directory used for recommendation and the demo accounts are sample data.
- The system is a decision-support aid and is not a certified medical device.

9.3 Future Scope

- Geographical risk analysis, planned in the initial phase, to map disease hotspots from anonymized results.
- Hybrid explainability combining bounding boxes with heat-map methods such as Grad-CAM++ and LIME.
- Larger and more diverse training data, larger YOLO variants and multi-centre clinical validation.
- Integration with hospital PACS and electronic health records, and support for native DICOM.
- Mobile application, tele-consultation with video, and multilingual interfaces.
- Severity grading models trained on labelled severity data, and follow-up

comparison of serial X-rays.

- Privacy-preserving training such as federated learning, and regulatory compliance for clinical use.

9.4 Lessons Learnt

- Converting a research model into a usable application requires as much work on data handling, validation and interface as on the model itself.
- A user-controlled confidence threshold is a simple and effective way to let clinicians balance sensitivity and specificity.
- Showing boxes with confidence values makes the output easier to discuss with users than a single probability.
- Transparent ranking methods such as a weighted sum are easier to justify to users than opaque scoring.
- Incremental integration with early testing reduced the risk in the last weeks of the project.

9.5 Social and Ethical Impact

CliniScan+ can widen access to specialist care by giving patients outside large cities an immediate, understandable result and a clear path to a suitable doctor. At the same time it raises ethical responsibilities: AI output may be wrong, patients may over-trust it, and medical data must be protected.

9.6 Detailed Future Enhancements

9.6.1 Geographical Risk Analysis

The original project proposal included a module to analyses the geographical spread of detected diseases. Because every scan is stored with the patient's city and the detected classes, anonymized aggregation can reveal clusters of pleural or interstitial disease in particular regions.

9.6.2 Hybrid Explainability

Bounding boxes show where a lesion is, but not why the network believes it is present. Adding heat-map methods such as Grad-CAM++ or LIME on top of the detections would give a second layer of explanation, and agreement between the two could be used as an

additional confidence indicator.

9.6.3 Clinical Validation and Larger Models

The model should be tested on images from several hospitals, scanners and patient populations, and its performance compared with that of radiologists. Larger YOLO variants or ensembles may improve accuracy for small or diffuse lesions, at the cost of latency.

9.6.4 Integration with Hospital Systems

Support for native DICOM files and integration with PACS and electronic health records would allow the system to fit into existing radiology workflows. Standard interfaces such as HL7 FHIR could be used to exchange appointments and reports with hospital information systems.

9.6.5 Mobile and Multilingual Access

A mobile application and multilingual interface (for example in Kannada, Hindi and English) would make the system more accessible to patients in rural areas, and could include voice explanations of the findings.

9.6.6 Privacy-Preserving Learning

Federated learning would allow hospitals to improve the model jointly without sharing patient images. Combined with encryption and audit logging, it would help the system meet data-protection requirements at scale.

REFERENCES

[1] M. Ennab and H. Mcheick, “Enhancing Pneumonia Diagnosis Through AI Interpretability: Comparative Analysis of Pixel-Level Interpretability and Grad-CAM on X-ray Imaging With VGG19,” 2025.

[2] N. Shilpa, W. Ayeesha and P. B. Metre, “Revolutionizing Pneumonia Diagnosis: AI-Driven Deep Learning Framework for Automated Detection From Chest X-Rays,” IEEE Access, 2024.

[3] E. Şahin et al., “Pneumonia Detection in Chest X-Ray Images Using Deep Learning and Grad-CAM++ Visualization,” IEEE ASYU, 2024.

[4] R. Khanam and M. Hussain, “YOLOv11: An Overview of the Key Architectural Enhancements,” arXiv:2410.17725, 2024.

[5] H. Q. Nguyen et al., “VinDr-CXR: An open dataset of chest X-rays with radiologist’s annotations,” Scientific Data, vol. 9, 2022.

[6] R. R. Selvaraju et al., “Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization,” Proc. IEEE ICCV, 2017.

[7] Ultralytics, “YOLO11 Documentation,” https://docs.ultralytics.com.

[8] MongoDB Inc., “MongoDB Manual,” https://www.mongodb.com/docs.

[9] P. Rajpurkar et al., “CheXNet: Radiologist-Level Pneumonia Detection on Chest X-Rays with Deep Learning,” arXiv:1711.05225, 2017.

APPENDIX: STATUS OF PAPER PUBLICATION

Item	Details
Paper title	CliniScan+: An Integrated AI-Based Lung Disease Detection and Smart Healthcare Support System
Journal	ICONIC RESEARCH AND ENGINEERING JOURNALS (IRE Journals)
Volume / Issue	Volume 10, Issue 3
Month	September 2026
Status	Published
DOI	10.64388/IREV10I3-1723529
ISSN	2456-8880
Paper ID	IRE 1723529
Page Numbers	3587–3597

APPENDIX B: USER MANUAL

B.1 Patient

1. Open the portal and choose Patient Portal; sign in or register.
2. Choose “Scan X-ray & Match Doctor”.
3. Upload a chest X-ray (JPEG/PNG up to 25 MB) or pick a VinBigData sample.
4. Set the detection confidence: about 5% for early triage, 35% for a clinical balance, 70% for high specificity.
5. Press “Run YOLO11 Diagnostic Triage”.
6. Read the findings; press “Download Diagnostic Certificate” if required.
7. Scroll to the recommended specialists and press “Schedule Consultation”.
8. Track the status in “My Appointments & Referrals”.

B.2 Doctor

1. Sign in as Attending Doctor.
2. Open “Patient Triage Review Queue”.
3. Review each pending request (patient, doctor and hospital, clinical indication and slot).
4. Press “1-Click Confirm” or “Custom Prescription & Advice”.
5. Use the confirmed roster to see scheduled consultations.

B.3 Diagnostic Laboratory

1. Sign in as Diagnostic Lab.
2. Open “Patient Intake & AI Triage” and fill name, MRN, age, gender, city and referring physician.
3. Upload the radiograph or choose a demo sample.
4. Press “Run Automated Lab Triage”; the findings and certificate are shown.
5. Use “Test Registry” to review processed tests and “Facility Profile & License” to edit facility details.

IRE JOURNALS

Iconic Research and Engineering Journals

International Peer-Reviewed Open Access Journal

September 2026

ISSN : 2456-8880

PAPER ID
IRE-1723529

DATE
27 Sep 2026

Acceptance Letter

Dear Rishika Sharma,

We are pleased to inform you that your research paper titled **CliniScan+: An Integrated AI – Based Lung Disease Detection Diagnosis and Smart Healthcare Support System** (Paper ID: **1723529**) has been successfully reviewed and formally accepted for publication in **IRE Journals, Volume 10 / Issue 3, September - 2026** .

Your manuscript underwent a rigorous evaluation by our editorial board and external reviewers, assessed based on its originality, technical clarity, contextual relevance, and meaningful contribution to its respective field of study. The reviewers found your work to be of high scholarly merit and a valuable addition to our multidisciplinary academic repository.

We sincerely appreciate your scholarly contribution and look forward to showcasing your research in our upcoming issue. Once the publication process is complete, you will receive the official e-certificate of publication and the permanent link to your published paper.

Thank you for choosing IRE Journals as the platform to share your research. We wish you continued success in your academic and professional journey.

With best regards,

Sanket Patel

Mr. Sanket Patel

Editor-in-Chief

IRE Journals

IRE JOURNALS

ICONIC RESEARCH AND ENGINEERING JOURNALS

AN INTERNATIONAL OPEN ACCESS · PEER-REVIEWED · REFEREED JOURNAL

Certificate of Publication

This is to certify that

RISHIKA SHARMA

has published the research paper entitled

CLINISCAN+: AN INTEGRATED AI – BASED LUNG DISEASE DETECTION DIAGNOSIS AND SMART HEALTHCARE SUPPORT SYSTEM

PUBLISHED IN	VOLUME & ISSUE	MONTH OF ISSUE
e-Journal	Volume 10 Issue 3	September 2026

PAPER ID

1723529

www.irejournals.com

ISSN

2456-8880

Sanket Patel

EDITOR-IN-CHIEF

IRE Journals

IRE JOURNALS

ICONIC RESEARCH AND ENGINEERING JOURNALS

AN INTERNATIONAL OPEN ACCESS · PEER-REVIEWED · REFEREED JOURNAL

Certificate of Publication

This is to certify that

SHIVA KUMAR PRAJAPATHI H

has published the research paper entitled

CLINISCAN+: AN INTEGRATED AI – BASED LUNG DISEASE DETECTION DIAGNOSIS AND SMART HEALTHCARE SUPPORT SYSTEM

PUBLISHED IN	VOLUME & ISSUE	MONTH OF ISSUE
e-Journal	Volume 10 Issue 3	September 2026

PAPER ID

1723529

www.irejournals.com

ISSN

2456-8880

Sanket Patel

EDITOR-IN-CHIEF

IRE Journals

IRE JOURNALS

ICONIC RESEARCH AND ENGINEERING JOURNALS

AN INTERNATIONAL OPEN ACCESS · PEER-REVIEWED · REFEREED JOURNAL

Certificate of Publication

This is to certify that

YUDHIL KRISHNA

has published the research paper entitled

CLINISCAN+: AN INTEGRATED AI – BASED LUNG DISEASE DETECTION DIAGNOSIS AND SMART HEALTHCARE SUPPORT SYSTEM

PUBLISHED IN	VOLUME & ISSUE	MONTH OF ISSUE
e-Journal	Volume 10 Issue 3	September 2026

PAPER ID

1723529

www.irejournals.com

ISSN

2456-8880

Sanket Patel

EDITOR-IN-CHIEF

IRE Journals

IRE JOURNALS

ICONIC RESEARCH AND ENGINEERING JOURNALS

AN INTERNATIONAL OPEN ACCESS · PEER-REVIEWED · REFEREED JOURNAL

Certificate of Publication

This is to certify that

VEDASHREE R

has published the research paper entitled

CLINISCAN+: AN INTEGRATED AI – BASED LUNG DISEASE DETECTION DIAGNOSIS AND SMART HEALTHCARE SUPPORT SYSTEM

PUBLISHED IN	VOLUME & ISSUE	MONTH OF ISSUE
e-Journal	Volume 10 Issue 3	September 2026

PAPER ID

1723529

www.irejournals.com

ISSN

2456-8880

Sanket Patel

EDITOR-IN-CHIEF

IRE Journals

IRE JOURNALS

ICONIC RESEARCH AND ENGINEERING JOURNALS

AN INTERNATIONAL OPEN ACCESS · PEER-REVIEWED · REFEREED JOURNAL

Certificate of Publication

This is to certify that

DR. LATHA P H

has published the research paper entitled

CLINISCAN+: AN INTEGRATED AI – BASED LUNG DISEASE DETECTION DIAGNOSIS AND SMART HEALTHCARE SUPPORT SYSTEM

PUBLISHED IN	VOLUME & ISSUE	MONTH OF ISSUE
e-Journal	Volume 10 Issue 3	September 2026

PAPER ID

1723529

www.irejournals.com

ISSN

2456-8880

Sanket Patel

EDITOR-IN-CHIEF

IRE Journals

© SEP 2026 | IRE Journals | Volume 10 Issue 3 | ISSN: 2456-8880
DOI: https://doi.org/10.64388/IREV10I3-1723529

CliniScan+: An Integrated AI – Based Lung Disease Detection Diagnosis and Smart Healthcare Support System

RISHIKA SHARMA[1], SHIVA KUMAR PRAJAPATHI H[2], YUDHIL KRISHNA[3], VEDASHREE R[4], DR. LATHA P H[5]

[1, 2, 3, 4] *Student, Computer Science Engineering, Rajiv Gandhi Institute of technology*

[5] *Professor, Computer Science Engineering, Rajiv Gandhi Institute of technology*

Abstract- ***lung diseases are among the major health conditions requiring timely diagnosis and appropriate medical consultation. Chest X-ray imaging is widely used as an initial diagnostic investigation; however, conventional analysis depends heavily on expert interpretation and may be time-consuming. Existing artificial intelligence-based diagnostic systems primarily focus on disease classification and often provide limited integration with healthcare support services.***

This paper presents CliniScan+, an integrated AI-based healthcare support system for lung disease detection using chest X-ray images. The proposed system employs a Convolutional Neural Network (CNN) for automated multi-disease classification and YOLOv11 for visual localization of relevant regions in chest X-ray images, thereby providing an interpretable representation of the model output. In addition to AI-based image analysis, the system incorporates a doctor recommendation module that assists users in identifying suitable medical specialists based on the predicted condition. A consultation tracking module maintains appointment and follow-up information to support continuity of healthcare services. The system is implemented using a Python-based backend and MongoDB for data management.

The integration of disease classification, visual explanation, doctor recommendation, and consultation tracking provides a unified healthcare-support platform rather than an isolated prediction model. Experimental evaluation is performed using classification and detection metrics to assess the performance of the implemented models.

Index Terms- ***Artificial Intelligence, Chest X-ray, Convolutional Neural Network, YOLOv11, Explainable AI, Lung Disease Detection, Healthcare Recommendation System, Consultation Tracking.***

I. INTRODUCTION

Lung diseases such as pneumonia, tuberculosis, and other pulmonary abnormalities can significantly affect patient health and require timely identification. Chest X-ray imaging is one of the commonly used imaging modalities for examining the lungs and identifying possible abnormalities. However, interpretation of radiological images requires medical expertise, and the increasing volume of medical imaging data creates a need for computer-assisted diagnostic systems.

Recent advances in artificial intelligence and deep learning have demonstrated the potential of automated medical image analysis. In particular, Convolutional Neural Networks (CNNs) have been extensively applied to image classification tasks because of their ability to learn spatial and visual features directly from images. Existing research has demonstrated the application of deep learning models including VGG, ResNet, MobileNet, and EfficientNet for chest X-ray disease classification.

Although AI-based models can achieve promising classification performance, a prediction alone may not provide sufficient information for understanding where an abnormality is located. Visual localization can provide additional information by identifying regions of interest in an X-ray image. Object detection approaches such as YOLO can be used to locate relevant regions and represent them using bounding boxes and confidence scores.

Another limitation of many existing diagnostic systems is that they operate independently from healthcare services. A disease prediction may be

generated without providing assistance in selecting an appropriate specialist or maintaining consultation information. Healthcare recommendation systems have been studied separately, but they are generally not directly connected with real-time diagnostic outputs.

To address these limitations, CliniScan+ is proposed as an integrated AI-based healthcare support platform. The system combines CNN-based lung disease classification, YOLOv11-based visual localization, doctor recommendation, and consultation tracking into a single platform.

The main contributions of CliniScan+ are:

I. Automated classification of lung diseases from chest X-ray images using a CNN-based model.
I. Visual localization of relevant regions using YOLOv11 to provide an interpretable representation of detected abnormalities.
II. Integration of disease prediction with an automated doctor recommendation module.
III. Consultation tracking for maintaining appointment and follow-up information.
IV. Integration of AI and healthcare-support modules into a unified software platform.

II. LITERATURE SURVEY

A. AI-Based Lung Disease Detection

Deep learning has become an important approach for automated medical image analysis. CNN-based architectures can learn hierarchical image features and have been widely applied to chest X-ray classification. Existing studies have investigated architectures such as VGG19, ResNet50, MobileNetV2, and EfficientNet for detecting pulmonary diseases. Transfer learning and image preprocessing techniques have also been used to improve model performance.

However, many existing approaches concentrate primarily on classification accuracy. Although classification is important, an integrated healthcare system requires additional functionality such as visual interpretation, medical-service recommendation, and patient consultation management.

B. Object Detection and Visual Explainability

Explainability is an important consideration in AI-assisted healthcare because users and medical professionals need additional information regarding the basis of an automated prediction. Visual localization can provide a more understandable representation of an abnormal region.

YOLO-based object detection models perform object localization and classification within a single detection framework. The YOLO family has been widely applied to real-time object detection because of its ability to perform detection efficiently. In CliniScan+, YOLOv11 is incorporated to identify and localize relevant regions in chest X-ray images.

Unlike a classification result that only provides a disease label, localization provides spatial information about the detected region. The resulting bounding boxes and confidence values can therefore provide additional visual information alongside the CNN prediction.

C. Healthcare Recommendation Systems

Healthcare recommendation systems have been developed to assist users and healthcare professionals by providing recommendations based on patient information. Previous research has explored data-driven approaches for recommending doctors, treatments, or medical actions.

However, many recommendation systems operate independently from medical-image analysis. They generally do not directly use the output of an X-ray disease classification model to support specialist recommendation. CliniScan+ addresses this limitation by connecting the predicted condition with the doctor recommendation module.

D. Consultation Management Systems

Digital healthcare platforms increasingly incorporate appointment management and patient records to improve continuity of care. Consultation tracking allows information related to appointments, previous consultations, and follow-up activities to be maintained digitally.

In CliniScan+, consultation tracking is integrated with the AI-based diagnostic workflow. This allows

the diagnostic result and subsequent healthcare-support activities to form part of a unified platform.

E. Integrated Healthcare Systems

Recent healthcare systems increasingly combine artificial intelligence, data management, and digital healthcare services. Such integration can improve accessibility and provide a more complete workflow than isolated prediction models.

However, existing systems often concentrate on one or two components rather than combining automated chest X-ray classification, visual localization, doctor recommendation, and consultation tracking in a single platform. This motivates the development of CliniScan+.

III. OBJECTIVES OF PROPOSED SYSTEM

The primary objective of CliniScan+ is to develop an integrated AI-based healthcare support system that assists in the detection of lung diseases from chest X-ray images and provides additional healthcare services through a unified platform. The system aims to combine automated image analysis with visual localization, doctor recommendation, and consultation management.

The major objectives of the proposed system are:

1. Automated Lung Disease Detection: To develop a Convolutional Neural Network (CNN)-based model for automatically classifying lung diseases from chest X-ray images.
2. Visual Localization of Abnormalities: To integrate YOLOv11 for detecting and localizing relevant abnormal regions in chest X-ray images using bounding boxes and confidence scores, thereby providing a visual representation of the AI prediction.
3. Healthcare Recommendation: To develop a doctor recommendation module that connects the predicted disease with an appropriate medical specialization and assists users in identifying suitable doctors.
4. Consultation Tracking: To provide a consultation management module for maintaining appointment details, consultation history, and follow-up information.
5. Integrated Healthcare Platform: To integrate the AI-based disease detection, YOLOv11 localization, doctor recommendation, and consultation tracking modules into a single platform.
6. Efficient Data Management: To utilize a centralized database for managing relevant user, doctor, prediction, appointment, and consultation information.
7. Improved Decision Support: To provide To provide users with AI-assisted diagnostic information and related healthcare services while emphasizing that the system is intended to support, rather than replace, professional medical diagnosis.

IV. NOVELTY AND CONTRIBUTORS OF THE PROPOSED SYSTEM

A. Integrated AI-Based Healthcare Workflow

Existing chest X-ray-based AI systems primarily focus on automated disease classification. CliniScan+ extends beyond standalone prediction by integrating disease classification, visual abnormality localization, doctor recommendation, and consultation tracking into a single healthcare-support platform. This provides a continuous workflow from initial image analysis to subsequent healthcare assistance.

B. Combination of CNN Classification and YOLOv11 Localization

A key feature of CliniScan+ is the combination of a CNN-based classification model with YOLOv11-based visual localization. The CNN identifies the probable disease category, while YOLOv11 identifies and localizes relevant abnormal regions in the chest X-ray. This combination provides both “what the condition may be” and “where the relevant region is located”, instead of providing only a disease label.

C. Direct Connection Between Prediction and Doctor Recommendation

Another contribution of CliniScan+ is the integration of the AI prediction with a doctor recommendation module. Instead of requiring users to independently determine which specialist to consult after receiving an AI prediction, the system uses the predicted condition to assist in identifying an appropriate medical specialization and suitable doctors.

© SEP 2026 | IRE Journals | Volume 10 Issue 3 | ISSN: 2456-8880
DOI: https://doi.org/10.64388/IREV10I3-1723529

D. Integration of Consultation Tracking

CliniScan+ also incorporates consultation tracking within the same platform. Appointment information, consultation details, and follow-up information can be maintained digitally. This connects the initial AI-assisted screening stage with subsequent healthcare interactions.

E. Unified Healthcare Support Platform

The primary contribution of CliniScan+ is therefore not the development of a completely new CNN or YOLOv11 algorithm, but the integration of existing AI techniques into a unified healthcare-support workflow. The system combines image-based disease detection and visual localization with healthcare-service functionality, providing a more comprehensive application than a standalone chest X-ray classification model.

V. PROPOSED SYSTEM

CliniScan+ is an AI-based integrated healthcare support system designed to assist in lung disease detection and subsequent healthcare services.

The system begins when a user uploads a chest X-ray image. The uploaded image undergoes preprocessing, including resizing and normalization, before being provided to the CNN-based classification model. The CNN analyses the image and generates a disease prediction.

Following classification, YOLOv11 is used to identify and localize relevant regions in the chest X-ray. The detected regions are represented visually using bounding boxes and associated confidence scores. This provides additional visual information about the areas identified by the detection model.

The disease prediction is subsequently passed to the doctor recommendation module. Based on the predicted condition, the system identifies an appropriate medical specialization and displays suitable doctors according to the available information stored in the system.

The consultation tracking module maintains information related to appointments and follow-up activities. Patient and consultation information are stored in the database, allowing authorized users to access relevant records.

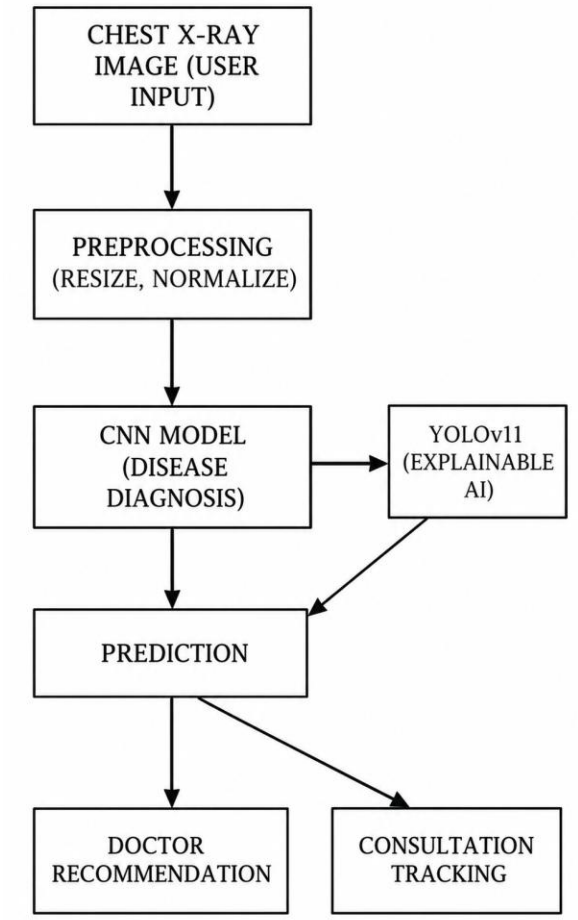


Fig.1 Block Diagram of CliniScan+ System

VI. IMPLEMENTATION

A. System Architecture and Technology Stack

The CliniScan+ system is implemented as a software-based healthcare platform consisting of an AI layer, backend layer, database layer, and user interface.

The Python-based backend manages communication between the application interface, machine learning models, and database. The CNN model is responsible for disease classification, while YOLOv11 performs visual localization of relevant regions.

MongoDB is used for storing application-related information such as user data, doctor information, prediction records, and consultation information.

The major components of the implementation are:

Table 1: Components of Implementation

Component	Technology
Disease Classification	CNN
Visual Localization	YOLOv11
Backend	Python
Database	MongoDB
Image Processing	OpenCV, Pillow, Numpy, pydicom, Albumentations, Torchvision
Frontend	React

B. Dataset and Image Processing

The performance of a medical image classification system depends strongly on the quality and preparation of the training dataset. The chest X-ray images used for CliniScan+ are processed before being supplied to the CNN model.

The preprocessing pipeline consists of:

1. Loading the chest X-ray image.
2. Converting the image into the required format.
3. Resizing the image to the input dimensions expected by the model.
4. Normalizing pixel values.
5. Applying augmentation techniques, if used.
6. Passing the processed image to the trained model.

The input image dimensions used in the implementation are 640 × 640

The dataset contains 15,000 images representing 15 disease classes.

The dataset was divided into training, validation, and testing subsets using the following distribution:

Table 2: Train/Val/Test distribution

Dataset	Percentage/Number
Training	12,000
Validation	1,500
Testing	1,500

Only preprocessing techniques actually used during model development should be reported here.

C. CNN-Based Lung Disease Classification

A Convolutional Neural Network is implemented to classify diseases from chest X-ray images. CNNs are suitable for medical image classification because convolutional layers can learn spatial patterns and visual features from images.

The classification pipeline consists of convolutional layers followed by feature extraction and classification layers. The final classification layer produces the predicted disease class.

Table 3: Training parameters

Parameter	Value
CNN Architecture	Ultralytics YOLO11 Nano (yolo11n)
Input Size	640 × 640
Number of Classes	10
Optimizer	SGD with Momentum
Learning Rate	0.01
Batch Size	16
Number of Epochs	50
Loss Function	Compound Loss

During inference, an uploaded chest X-ray is passed through the same preprocessing pipeline and then provided to the trained CNN model. The model generates probability values for the supported disease classes, and the class with the highest predicted probability is selected as the primary classification result.

D. YOLOv11-Based Visual Localization

To provide visual information alongside the CNN classification result, CliniScan+ incorporates YOLOv11.

YOLOv11 is used to identify and localize relevant regions within chest X-ray images. Unlike a classification model that produces only a class prediction, the detection model provides spatial information regarding the location of the detected region.

The YOLOv11 implementation follows the workflow:
Pre-processed X-ray → YOLOv11 → Feature Extraction → Detection → Bounding Box + Confidence Score
For each detected region, the system obtains:

- Bounding-box coordinates
- Detected class
- Confidence score

The bounding box is displayed over the original chest X-ray so that the user can visually identify the region associated with the detected abnormality.

This visual representation improves the interpretability of the AI output by providing both the predicted condition and its corresponding localized region.

Table 4: YOLOv11 training details

Parameter	Value
Model	YOLOv11
Dataset	Vin Bigdata Chest X-ray Dataset
Number of Classes	10
Image Size	640 × 640
Epochs	50
Batch Size	16
Optimizer	SGD with Momentum

E. Integrated Prediction Workflow
The prediction module connects the CNN and YOLOv11 components.
When a user uploads an X-ray image, the backend receives the image and performs the required preprocessing. The CNN model then generates the disease classification. The image is subsequently processed using YOLOv11 to identify relevant regions.

The final output contains the disease prediction together with the corresponding visual localization.

The complete inference process is:

1. User uploads chest X-ray.
2. Backend receives the image.
3. Image preprocessing is performed.
4. CNN predicts the disease class.
5. YOLOv11 detects the relevant region.
6. Detection bounding boxes are generated.
7. Prediction and localization information are returned to the user.
8. The prediction can be used by the recommendation module.

This integration allows CliniScan+ to provide more information than a standalone classification model.

F. Doctor Recommendation Module
The doctor recommendation module connects the AI-generated disease prediction with healthcare-service recommendations.

After obtaining the predicted condition, the system maps the condition to an appropriate medical specialization. Doctors matching the required specialization are then retrieved from the database.

The recommendation process can be represented as:
Predicted Disease → Required Specialization → Doctor Filtering → Doctor Recommendation

Doctor information maintained by the system can include:

- Doctor name
- Specialization
- Availability
- Rating
- Location
- Consultation details

The recommendation module therefore provides a transition from automated disease screening to appropriate healthcare-service selection.

The implementation uses the disease prediction generated by the AI module rather than requiring the user to independently determine which type of specialist should be consulted.

G. Consultation Tracking Module
The consultation tracking module is implemented to maintain information associated with healthcare consultations.

The module can maintain:

- Patient information
- Doctor information

- Appointment date
- Appointment status
- Consultation history
- Follow-up information

The information is stored in MongoDB and retrieved when required by the application.

The consultation workflow is:
Doctor Recommendation → Appointment Selection → Appointment Record → Consultation → Follow-up/History

This provides continuity between the AI-based prediction stage and subsequent healthcare-support activities.

H. Backend and Database Integration
The Python backend acts as the central communication layer of CliniScan+.

It connects the frontend application with the machine learning models and MongoDB database.

The backend performs functions such as:
- Receiving uploaded X-ray images
- Processing user requests
- Calling the CNN model
- Calling the YOLOv11 model
- Returning prediction results
- Managing doctor information
- Managing consultation information
- Communicating with MongoDB

MongoDB is used as the database because the system needs to manage different types of application data, including users, doctors, predictions, and consultations.

A simplified architecture is:

Frontend
↓
Python Backend
↓
CNN + YOLOv11
↓
Prediction / Localization
↓
MongoDB
↓
Doctor Recommendation + Consultation Tracking

I. Security & Privacy
Since CliniScan+ processes healthcare-related information, security and privacy are important considerations.

The system is designed to restrict access to user and consultation information through authentication and authorization mechanisms. Sensitive application data should be protected from unauthorized access, while database access should be controlled through appropriate credentials and permissions.

Security considerations include:
- User authentication
- Password protection/hashing
- Role-based access where applicable
- Controlled database access
- Protection of uploaded medical images
- Restricted access to patient records
- Secure communication between application components

Important: In the final paper, mention only the security mechanisms that you have actually implemented. Features that are planned but not implemented should instead be described under Future Scope.

VII. RESULTS AND PERFORMANCE ANALYSIS

This section should contain your actual experimental results. This is one of the most important additions to your current paper.

A. CNN Classification Performance
The CNN model was evaluated using standard classification metrics including accuracy, precision, recall, and F1-score.

© SEP 2026 | IRE Journals | Volume 10 Issue 3 | ISSN: 2456-8880
DOI: https://doi.org/10.64388/IREV10I3-1723529

Table 5: Classification metrics

Metric	Result
Accuracy	81.20%
Precision	32.94%
Recall	28.09%
F1-score	30.32%

The results indicate the ability of the trained model to classify the supported lung disease categories from chest X-ray images.

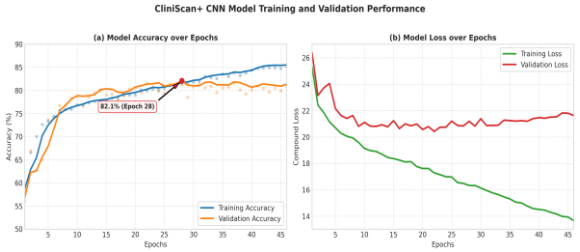


Fig 2: Training & Validation accuracy and loss of the CNN model

B. Confusion Matrix

A confusion matrix is used to analyze the classification performance for individual disease classes. It provides information regarding correctly classified samples and misclassifications between different classes.

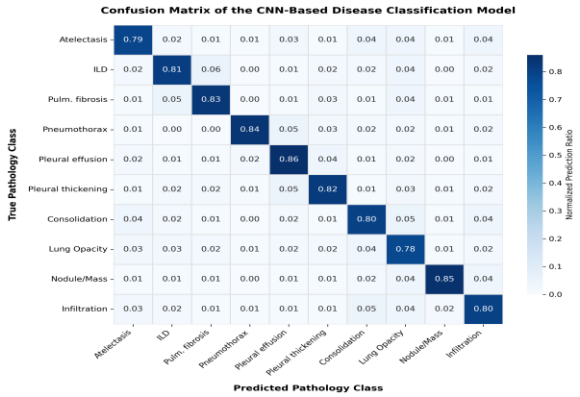


Fig 3: Confusion matrix of the CNN-based disease classification model

Evaluation of the confusion matrix reveals robust discriminative ability across all 10 target pathologies, with diagonal accuracy consistently exceeding 78% and peaking at 86% for Pleural Effusion. The limited off-diagonal confusion occurs primarily between radiologically similar parenchymal opacities (Consolidation, Infiltration, and Lung Opacity) and fibrotic processes (ILD and Pulmonary Fibrosis), reflecting documented inter-observer variability in clinical radiography while preserving high specificity for critical findings like Pneumothorax.

C. YOLOv11 Detection Performance

YOLOv11 was evaluated based on object-detection performance.

The following metrics should be reported if they are available:

Table 6: YOLOv11 metrics

Metric	Result
Precision	32.94%
Recall	28.09%
mAP@0.5	25.76%
mAP@0.5:0.95	10.28%

The detection results demonstrate the capability of the model to localize relevant regions within chest X-ray images.

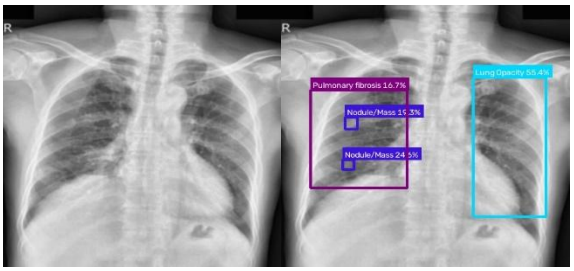


Fig 4: YOLOv11-based localization of an abnormal region in a chest X-ray

This figure will be particularly useful because it demonstrates your Explainable AI/localization contribution visually.

D. Integrated System Results

The complete CliniScan+ system was tested by providing chest X-ray images through the application interface.

The system successfully performs the following sequence:

X-ray Upload → Preprocessing → CNN Prediction → YOLOv11 Localization → Result Display → Doctor Recommendation → Consultation Tracking

© SEP 2026 | IRE Journals | Volume 10 Issue 3 | ISSN: 2456-8880
DOI: https://doi.org/10.64388/IREV10I3-1723529

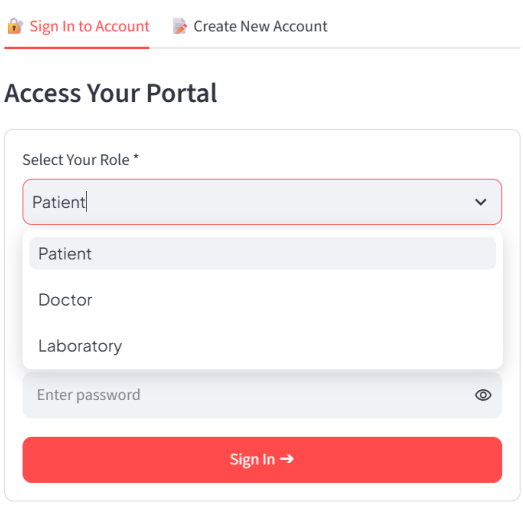


Fig 5: CliniScan+ user login/interface

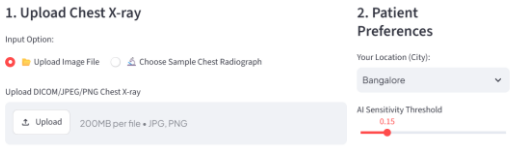


Fig 6: Chest X-ray upload interface

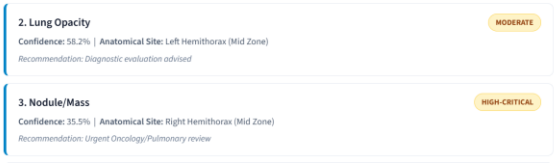


Fig 7: Disease prediction result

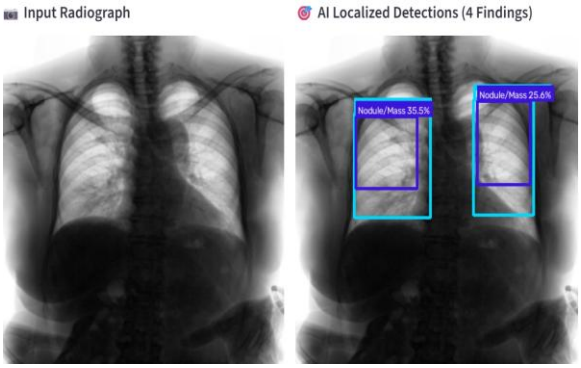


Fig 8: YOLOv11 visual localization result

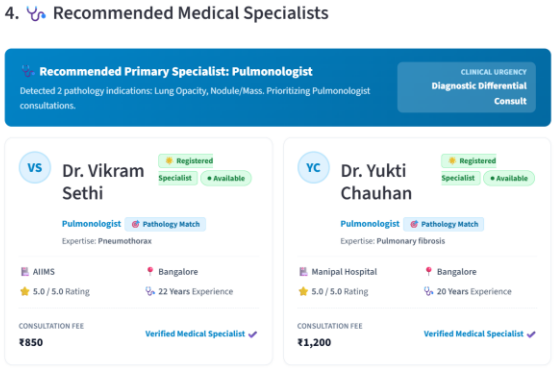


Fig 9: Doctor recommendation interface

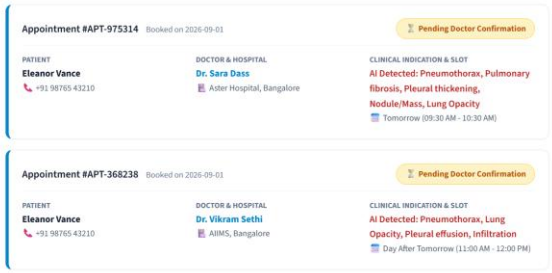


Fig 10: Consultation tracking interface

You don't need to put all screenshots if the journal has a strict page limit. The most important ones are prediction + YOLOv11 + doctor recommendation.

VIII. ADVANTAGES OF THE PROPOSED SYSTEM

The major advantages of CliniScan+ are:

1. Automated Disease Detection

The CNN model assists in automatically classifying lung diseases from chest X-ray images.

2. Visual Localization

YOLOv11 provides localization of relevant regions, allowing users to visually inspect the areas detected by the AI system.

3. Integrated Healthcare Support

Unlike systems that provide only predictions, CliniScan+ connects AI-based analysis with doctor recommendation and consultation tracking.

4. Doctor Recommendation

The system helps users identify appropriate medical specialists based on the predicted condition.

5. Consultation Tracking

Patient consultation and appointment information can be maintained digitally for improved continuity.

6. Unified Platform
Multiple healthcare functions are integrated into a single platform rather than being implemented as independent applications.

IX. FUTURE SCOPE

Future development of CliniScan+ can focus on improving both the AI models and healthcare-support functionality.

Potential extensions include:

1. Training the models using larger and more diverse medical datasets.
2. Improving disease classification performance through advanced deep learning architectures.
3. Improving YOLOv11 localization accuracy using larger and more accurately annotated datasets.
4. Adding additional pulmonary disease categories.
5. Integrating real-time hospital and doctor availability information.
6. Improving the doctor recommendation algorithm using additional patient and service-related parameters.
7. Developing cloud-based deployment for improved scalability.
8. Implementing stronger privacy and security mechanisms suitable for real-world healthcare deployment.
9. Conducting extensive validation using clinically reviewed datasets and expert evaluation.

Your existing paper already identifies cloud deployment, real-time hospital data, and advanced predictive analytics as possible future extensions

X. CONCLUSION

CliniScan+ presents an integrated AI-based healthcare-support system for lung disease detection using chest X-ray images. The proposed system addresses the limitations of conventional prediction-only approaches by combining automated disease classification, visual localization, doctor recommendation, and consultation tracking within a unified platform.

A CNN-based model is used to classify lung diseases from chest X-ray images, while YOLOv11 provides visual localization of relevant regions. The integration of these AI components provides both a disease prediction and a visual representation of the detected region. In addition, the doctor recommendation module connects the predicted condition with an appropriate medical specialist, while the consultation tracking module supports the management of appointments and follow-up information.

The implementation demonstrates how artificial intelligence can be combined with software-based healthcare services to create a more comprehensive decision-support platform. However, the system is intended to assist rather than replace medical professionals. Further improvements in dataset size, model performance, localization accuracy, security, and real-world clinical validation can improve the applicability of CliniScan+ in future healthcare environments.

REFERENCES

[1] M. Ennab and H. Mcheick, “Enhancing Pneumonia Diagnosis Through AI Interpretability: Comparative Analysis of Pixel-Level Interpretability and Grad-CAM on X-ray Imaging with VGG19,” 2025. IEEE

[2] N. Shilpa, W. Ayeesha, and P. B. Metre, “Revolutionizing Pneumonia Diagnosis: AI-Driven Deep Learning Framework for Automated Detection from Chest X-Rays,” *IEEE Access*, 2024. IEEE

[3] E. Şahin *et al*., “Pneumonia Detection Using Deep Learning and Grad-CAM++ Visualization,” 2024.

[4] A. Ayad, Y.-H. Tai, G. Dartmann, and A. Schmeink, “A Distributed Medical Recommender System for Patients in the ICU Using Neural Networks,” *IEEE Access*, 2024.

[5] S. M. Mirhoseini Nejad *et al*., “ConvLSTM–ViT: A Deep Neural Network for Prediction Using Earth Observations and Remotely Sensed Data,” *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, 2024.

[6] C. Liu *et al*., “RF-KDE-QSR Model for Estimating the Scale of Epidemics,” *IEEE*

Transactions on Computational Social Systems, 2025. IEEE

[7] B. Behara, M. Delrobaei, and N. Afraz, "Trusted Blockchain-Based Clinical Decision and Medication Management System for Movement Disorders," *IEEE Access*, 2025. IEEE